

Red Hat Lightspeed cost management API

Pau Garcia Quiles, Senior Principal Product Manager, Red Hat Lightspeed

Table of Contents

Introduction . . . 1
Authentication . . . 1
- Using service account tokens (OAuth2 client credentials) . . . 1

OpenShift Costs . . . 1
- Cluster costs . . . 1
- Project costs . . . 2
- Project costs grouped by cluster . . . 2
- Project costs grouped by tag . . . 2
- Node costs . . . 2
- Tag costs . . . 2
- OpenShift Virtualization VMs . . . 2
- CPU . . . 2
- Memory . . . 2
- Storage volumes . . . 3
- GPU costs . . . 3
- Breakdown page: costs for a cluster . . . 3
- Breakdown page: costs for a project . . . 3

Cloud Filtered by OpenShift . . . 4
AWS Costs . . . 4
- Per AWS account . . . 4
- Per AWS region . . . 4
- Per AWS service . . . 4
- Per AWS EC2 VM . . . 4
- Per AWS tag . . . 4
- Per AWS cost category . . . 4
- Per AWS Organizational Unit . . . 5
- Per AWS instance type . . . 5
- Per AWS storage . . . 5
- AWS cost types . . . 5

Azure Costs . . . 5
- Per Azure account (subscription) . . . 5
- Per Azure region . . . 5
- Per Azure service . . . 5
- Per Azure tag . . . 6
- Per Azure instance type . . . 6
- Per Azure storage . . . 6

Google Cloud Costs . . . 6
- Per Google Cloud account . . . 6
- Per Google Cloud region . . . 6
- Per Google Cloud service . . . 6
- Per Google Cloud Project . . . 6
- Per Google Cloud tag . . . 6

- Per Google Cloud instance type . . . 7
- Per Google Cloud storage . . . 7

Tags . . . 7
- List tag keys and values . . . 7

Optimizations (Resource Optimization for OpenShift) . . . 7
- Notification codes (reference) . . . 8
- Fleet summary . . . 8
- Container recommendations . . . 12
- Namespace recommendations . . . 21
- GPU recommendations hub (summary) . . . 23
- Node CPU/memory utilization recommendations . . . 27
- MachineSet aggregation recommendations . . . 28
- PVC (Persistent Volume Claim) recommendations . . . 29
- ResourceQuota recommendations . . . 32
- ClusterResourceQuota recommendations . . . 35
- Snapshot staleness recommendations . . . 38
- OpenShift Virtualization (VM) recommendations . . . 43
- Recommendation term settings . . . 53
- Settings capabilities . . . 53
- Platform settings lock . . . 54
- Recommendation threshold settings . . . 54
- Business hours settings . . . 58
- Idle and zombie detection . . . 59
- Recommendation history and quality . . . 63

Forecasts . . . 63
- OpenShift cost forecast . . . 63
- AWS cost forecast . . . 63
- Azure cost forecast . . . 63
- Google Cloud cost forecast . . . 63
- OCP-on-cloud cost forecasts . . . 64

Resource Types . . . 64
- OpenShift . . . 64
- AWS . . . 64
- Azure . . . 64
- Google Cloud . . . 64
- Cost models . . . 64

Cost Distribution Fields . . . 65

Settings . . . 65
- Platform projects (cost groups) . . . 65
- Enabled tags . . . 65
- Display currency . . . 65
- Cost models . . . 66

Integrations . . . 66

Filtering and Pagination . . . 66

Time scope filters . . . 66
Exclude filter . . . 67
Pagination . . . 67
Get CSV or JSON Reports . . . 67
Basic Field Definitions . . . 68

Introduction

The Cost Management API allows users to obtain all the data available in the UI, and even more since with the API it's possible to combine filters or to make several API calls and then combine the results.

The latest version of this cheat sheet, and its accompanying Bruno collection, are always available from `https://github.com/project-koku/costmgmt-api-cheatsheet`.

Below are common scenarios with example API calls.

Base URL: `https://console.redhat.com`

Authentication

Using service account tokens (OAuth2 client credentials)

Grab your token using the following curl command:

```
curl -v \
  -d "client_id=CLIENT_ID" \
  -d "client_secret=CLIENT_SECRET" \
  -d "grant_type=client_credentials" \
  -d "scope=api.console" \
  "https://sso.redhat.com/auth/realms/redhat-external/protocol/openid-connect/token"
```

Use the token in an API request via curl:

```
curl --location \
  'https://console.redhat.com/api/cost-management/v1/reports/aws/costs/' \
  --header 'Authorization: Bearer ACCESS_TOKEN'
```

OpenShift Costs

Cluster costs

```
GET /api/cost-management/v1/reports/openshift/costs/?group_by[cluster]=*
```

Cluster costs only for the current month:

```
GET /api/cost-
management/v1/reports/openshift/costs/?filter[time_scope_units]=month&group_by[cluster]=*
```

Project costs

```
GET /api/cost-management/v1/reports/openshift/costs/?group_by[project]=*
```

Project costs grouped by cluster

```
GET /api/cost-management/v1/reports/openshift/costs/?group_by[cluster]=*&group_by[project]=*
```

Project costs grouped by tag

Costs by project, grouped by a tag key (e.g. `application`):

```
GET /api/cost-
management/v1/reports/openshift/costs/?group_by[project]=*&group_by[tag:application]=*
```

Node costs

```
GET /api/cost-management/v1/reports/openshift/costs/?group_by[node]=*
```

Tag costs

Costs grouped by a tag key (e.g. `env`):

```
GET /api/cost-management/v1/reports/openshift/costs/?group_by[tag:env]=*
```

OpenShift Virtualization VMs

```
GET /api/cost-management/v1/reports/openshift/resources/virtual-machines/
```

Optionally filter by cluster:

```
GET /api/cost-management/v1/reports/openshift/resources/virtual-
machines/?filter[exact:cluster]=CLUSTER_UUID
```

CPU

```
GET /api/cost-management/v1/reports/openshift/compute/
```

Memory

```
GET /api/cost-management/v1/reports/openshift/memory/
```

Storage volumes

```
GET /api/cost-management/v1/reports/openshift/volumes/?group_by[storageclass]=*
```

Storage volumes for a specific project (e.g. `cost-management`):

```
GET /api/cost-management/v1/reports/openshift/volumes/?filter[exact:project]=cost-
management&group_by[storageclass]=*
```

GPU costs

```
GET /api/cost-management/v1/reports/openshift/gpu/
```

Group by node and project (common for MIG instance breakdown):

```
GET /api/cost-management/v1/reports/openshift/gpu/?group_by[node]=*&group_by[project]=*
```

GPU costs by MIG profile:

```
GET /api/cost-management/v1/reports/openshift/gpu/mig_profiles/
```

Group by MIG profile (requires `filter[gpu_vendor]`, `filter[gpu_model]`, and `filter[node]`):

```
GET /api/cost-management/v1/reports/openshift/gpu/mig_profiles/?group_by[mig_profile]=*
```

Breakdown page: costs for a cluster

Cost breakdown for the "demolab" cluster:

```
GET /api/cost-management/v1/reports/openshift/costs/?group_by[cluster]=demolab
```

Breakdown page: costs for a project

Cost breakdown for the "cost-management" project:

```
GET /api/cost-management/v1/reports/openshift/costs/?group_by[project]=cost-management
```

Cloud Filtered by OpenShift

The following endpoints return cloud data specifically relating to OpenShift clusters. They allow for both cluster and cloud group_by/filtering options (cluster, node, project, account, service, region).

```
GET /api/cost-management/v1/reports/openshift/infrastructures/all/costs/
GET /api/cost-management/v1/reports/openshift/infrastructures/aws/costs/
GET /api/cost-management/v1/reports/openshift/infrastructures/azure/costs/
GET /api/cost-management/v1/reports/openshift/infrastructures/gcp/costs/
```

AWS Costs

Per AWS account

```
GET /api/cost-management/v1/reports/aws/costs/?group_by[account]=*
```

Per AWS region

```
GET /api/cost-management/v1/reports/aws/costs/?group_by[region]=*
```

Per AWS service

```
GET /api/cost-management/v1/reports/aws/costs/?group_by[service]=*
```

Per AWS EC2 VM

```
GET /api/cost-management/v1/reports/aws/resources/ec2-compute/
```

Filtering by operating system (e.g. Fedora):

```
GET /api/cost-management/v1/reports/aws/resources/ec2-compute/?filter[operating_system]=Fedora
```

Per AWS tag

Costs grouped by an AWS tag key (e.g. `AppTeam`):

```
GET /api/cost-management/v1/reports/aws/costs/?group_by[tag:AppTeam]=*
```

Per AWS cost category

Costs grouped by an AWS cost category (e.g. `CostCenter`):

```
GET /api/cost-management/v1/reports/aws/costs/?group_by[aws_category:CostCenter]=*
```

Per AWS Organizational Unit

Group by root organizational unit "r-f22a":

```
GET /api/cost-management/v1/reports/aws/costs/?group_by[org_unit_id]=r-f22a
```

Per AWS instance type

```
GET /api/cost-management/v1/reports/aws/instance-types/
```

Per AWS storage

```
GET /api/cost-management/v1/reports/aws/storage/
```

AWS cost types

Getting unblended, blended and amortized costs:

```
GET /api/cost-management/v1/reports/aws/costs/?cost_type=unblended_cost
GET /api/cost-management/v1/reports/aws/costs/?cost_type=blended_cost
GET /api/cost-management/v1/reports/aws/costs/?cost_type=savingsplan_effective_cost
```

Azure Costs

Per Azure account (subscription)

```
GET /api/cost-management/v1/reports/azure/costs/?group_by[subscription_guid]=*
```

Per Azure region

```
GET /api/cost-management/v1/reports/azure/costs/?group_by[resource_location]=*
```

Per Azure service

```
GET /api/cost-management/v1/reports/azure/costs/?group_by[service_name]=*
```

Per Azure tag

Costs grouped by an Azure tag key (e.g. `ms-resource-usage`):

```
GET /api/cost-management/v1/reports/azure/costs/?group_by[tag:ms-resource-usage]=*
```

Per Azure instance type

```
GET /api/cost-management/v1/reports/azure/instance-types/
```

Per Azure storage

```
GET /api/cost-management/v1/reports/azure/storage/
```

Google Cloud Costs

Per Google Cloud account

```
GET /api/cost-management/v1/reports/gcp/costs/?group_by[account]=*
```

Per Google Cloud region

```
GET /api/cost-management/v1/reports/gcp/costs/?group_by[region]=*
```

Per Google Cloud service

```
GET /api/cost-management/v1/reports/gcp/costs/?group_by[service]=*
```

Per Google Cloud Project

```
GET /api/cost-management/v1/reports/gcp/costs/?group_by[gcp_project]=*
```

Per Google Cloud tag

Costs grouped by a Google Cloud label (e.g. `env`):

```
GET /api/cost-management/v1/reports/gcp/costs/?group_by[tag:env]=*
```

Per Google Cloud instance type

```
GET /api/cost-management/v1/reports/gcp/instance-types/
```

Per Google Cloud storage

```
GET /api/cost-management/v1/reports/gcp/storage/
```

Tags

List tag keys and values

Cloud source tags:

```
GET /api/cost-management/v1/tags/aws/
GET /api/cost-management/v1/tags/azure/
GET /api/cost-management/v1/tags/gcp/
```

OpenShift tags (labels):

```
GET /api/cost-management/v1/tags/openshift/
```

You can also do OCP-on-cloud variations by adding `infrastructures/{cloud}` (i.e. `gcp`, `azure`, `aws`).

Additional filtering options

Show disabled tags:

```
GET /api/cost-management/v1/tags/openshift/?filter[enabled]=False
```

Show tags for a particular time frame:

```
GET /api/cost-management/v1/tags/openshift/?start_date=2025-11-15&end_date=2025-12-17
```

See tag values directly for a specific key by appending the key ("my-tag-key") to the URL:

```
GET /api/cost-management/v1/tags/openshift/my-tag-key/
```

Optimizations (Resource Optimization for OpenShift)

Notification codes (reference)

Machine-readable catalog of all notification codes (no auth required). Optional `filter[plugin]` for `container`, `namespace`, `node`, `gpu`, `pvc`, `snapshot`, `vm`, `quota`, or `cluster-quota`.

```
GET /api/cost-management/v1/recommendations/openshift/notification-codes
GET /api/cost-management/v1/recommendations/openshift/notification-codes?filter[plugin]=container
GET /api/cost-management/v1/recommendations/openshift/notification-codes?filter[plugin]=namespace
```

Code **77** (`SPARSE_DATA`, severity INFO) — **Recommendation based on limited data; accuracy improves with more observation time** — is emitted on container, namespace, node, and PVC recommendations when `data_days` is at or below `sparse_data_threshold` (default **2**). It does not apply to GPU recommendations.

Fleet summary

Cross-cluster aggregated savings, adoption rates, and top optimization opportunities:

```
GET /api/cost-management/v1/recommendations/openshift/fleet-summary
```

Fleet savings summary

Aggregated monthly savings by plugin (container, node, PVC, snapshot, VM). Container, node, and VM totals honor `?engine=cost` (default) or `performance`. GPU dollar estimates are computed at API read time and are excluded from this rollup (`by_plugin.gpu` is always `0`; see `gpu_savings_note`).

Use `?term=short`, `?term=medium` (default), or `?term=long` to align node/container/PVC rollups with the same term window as list APIs. VM savings use mapped term keys (`short_term`, `medium_term`, `long_term`).

Requires persisted recommendations for the selected term. VM rows need `ROS_SAVINGS_ESTIMATES_ENABLED=true` and `KOKU_MASU_URL` at ingestion time for non-zero `by_plugin.vm`.

```
GET /api/cost-management/v1/recommendations/openshift/savings-summary
GET /api/cost-management/v1/recommendations/openshift/savings-summary?engine=cost
GET /api/cost-management/v1/recommendations/openshift/savings-summary?engine=cost&term=short
GET /api/cost-management/v1/recommendations/openshift/savings-summary?filter[cluster]=<cluster-
uuid>&group_by[tag:environment]=*
GET /api/cost-management/v1/recommendations/openshift/savings-summary?group_by[tag:environment]=*
GET /api/cost-management/v1/recommendations/openshift/savings-
summary?group_by[tag:environment]=*&filter[project]=payments
```

NOTE	`filter[cluster]` is only applied when a `group_by` parameter is present (e.g., `group_by[tag:*]` or `group_by[idle_state]`). `filter[project]` scopes container savings in tag-grouped responses only (requires `group_by[tag:*]`). Without `group_by`, cluster and project filters have no effect and all RBAC-visible clusters are included.

PVC plugin totals in `by_plugin` honor `term` only (engine-agnostic). Snapshot totals are term-independent (always sum all snapshot recommendations regardless of `term`). Container, node, and VM totals honor both `engine` and

`term`.

Negative savings

When current resource requests are already below the recommended target, `estimated_monthly_savings.value` (containers, nodes, PVCs, VMs) can be **negative**. That indicates an **additional** monthly cost if the recommendation is adopted (upsize for reliability or performance), not a savings opportunity. The UI should label negative values as additional cost rather than "Savings: -$X/month".

Effective rates (Koku Masu)

ROS does not expose a standalone cost-rate API. During ingestion (and savings recalculation), ros-processor fetches rates from Koku Masu:

```
GET /api/cost-management/v1/effective_rates/?cluster_id=<cluster-id>&org_id=<org-
id>&start_date=YYYY-MM-DD&end_date=YYYY-MM-DD
```

Returns the cluster cost model rates used for savings: CPU/memory request and usage rates per hour, storage rates per month, `node_cost_per_month`, optional `gpu_cost_per_month`, markup, `distribution_type`, and per-namespace aggregates (`cost_model_cpu_cost`, `infrastructure_cost`, `distributed_cost`, etc.). ROS caches the response per cluster for the ingest cycle.

When rates are missing or Masu is unreachable, recommendations still persist but dollar fields are omitted or zero and notification code **25** (`NotifNoCostData`) is appended on container, node, and PVC rows. Fleet `savings-summary` exposes `by_cluster[].has_cost_data: false` when every recommendation in that cluster lacks cost data.

Requires `KOKU_MASU_URL` on ros-processor/ros-api and `ROS_SAVINGS_ESTIMATES_ENABLED=true` (default).

Fleet heatmap

CPU and memory utilization heatmap across all clusters, showing hourly resource usage patterns. Requires Visual Insights enabled (`ROS_VISUAL_INSIGHTS_ENABLED=true`).

```
GET /api/cost-management/v1/recommendations/openshift/fleet-heatmap
GET /api/cost-management/v1/recommendations/openshift/fleet-heatmap?metric=memory
GET /api/cost-management/v1/recommendations/openshift/fleet-heatmap?filter[cluster]=CLUSTER_UUID
```

Default metric is `cpu`. Use `?metric=memory` for memory heatmap. Response returns a matrix of hourly utilization data points across nodes.

Savings recalculation (internal)

After a Koku cost model update, masu notifies ROS to refresh persisted dollar savings without re-ingestion. Supported types: `container`, `node`, `pvc`, `quota`, `cluster-quota`. Service-account bearer token required (same auth as tag sync). Requires `ROS_SAVINGS_ESTIMATES_ENABLED` and `ROS_SAVINGS_RECALCULATION_ENABLED`.

OpenShift Virtualization (VM) monthly savings are **not** refreshed by this endpoint. VM `estimated_monthly_savings`

on list/detail and `by_plugin.vm` in `GET /savings-summary` are computed during VM recommendation ingestion when `ROS_SAVINGS_ESTIMATES_ENABLED=true` and `KOKU_MASU_URL` is available; a new ingest cycle is required after cost model rate changes.

Plugin / type	`recalculate-savings` support
`container`, `node`, `pvc`, `quota`, `cluster-quota`	Supported – pass in `recommendation_types`
`container` (GPU MIG/idle)	Included when `container` is in `recommendation_types` – refreshes `estimated_gpu_savings_cents`
`vm`	N/A – computed at ingestion (not via recalculate)
`snapshot`	N/A – snapshot totals are term-independent; uses settings/env rate chain
`gpu` (time-slicing)	N/A – persisted as `estimated_savings_cents` at ingest (not via recalculate); run ingest or [Backfill GPU time-slicing] after rate changes

```
POST /api/cost-management/v1/internal/recalculate-savings
```

Example body:

```
{
  "org_id": "1234567",
  "cluster_uuid": "02059694-68ab-4d58-8809-de1e91f1d0e5",
  "recommendation_types": ["container", "node", "pvc", "quota", "cluster-quota"]
}
```

Returns `202 Accepted` with `status: "accepted"`; work runs asynchronously.

Backfill GPU time-slicing (internal)

Populate `node_gpu_timeslicing_recommendations` for orgs that already have GPU digest data but no persisted time-slicing rows (for example, after enabling persistence on an existing deployment). Service-account bearer token required (same auth as tag sync). Requires the `gpu` plugin.

```
POST /api/cost-management/v1/internal/backfill-gpu-timeslicing
POST /api/cost-management/v1/internal/backfill-gpu-timeslicing?org_id=1234567
POST /api/cost-management/v1/internal/backfill-gpu-
timeslicing?org_id=1234567&cluster_uuid=02059694-68ab-4d58-8809-de1e91f1d0e5
```

Optional query params: `org_id`, `cluster_uuid` (omit both to backfill all orgs/clusters). Returns `200 OK` with `status: "completed"`, `orgs_processed`, and `clusters_processed`.

Example response excerpt:

```
{
```

```
  "status": "completed",
  "org_id": "1234567",
  "cluster_uuid": "02059694-68ab-4d58-8809-de1e91f1d0e5",
  "orgs_processed": 1,
  "clusters_processed": 1
}
```

Tag sync status (internal)

Per-org tag catalog metadata: which tag keys are known and the last sync time. Service-account bearer token required (same auth as tag sync). Requires `ROS_TAGS_ENABLED`.

```
GET /api/cost-management/v1/internal/tags/status?org_id=1234567
```

Required: `org_id`. Response includes `tag_keys` (array of known keys), `source` (`"api"` or `"db"`), and `synced_at` timestamp (API mode only).

Savings summary response excerpt:

```
{
  "currency": "USD",
  "estimated_monthly_savings": { "value": "12500.75", "units": "USD" },
  "by_plugin": {
    "container": { "value": "5000.00", "units": "USD" },
    "node": { "value": "3000.00", "units": "USD" },
    "pvc": { "value": "1500.00", "units": "USD" },
    "snapshot": { "value": "500.00", "units": "USD" },
    "vm": { "value": "1200.00", "units": "USD" },
    "gpu": { "value": "0.00", "units": "USD" }
  },
  "gpu_savings_note": "GPU savings are computed at API read time..."
}
```

Workload types (metadata)

Returns the distinct Kubernetes owner-kind values (e.g., `Deployment`, `StatefulSet`, `DaemonSet`) observed for the authenticated org. Useful for populating workload-type filter dropdowns in the UI.

```
GET /api/cost-management/v1/recommendations/openshift/workload-types
```

Response:

```
{
  "data": ["DaemonSet", "Deployment", "StatefulSet"]
}
```

Container recommendations

```
GET /api/cost-management/v1/recommendations/openshift/
```

Recommendations for a specific container (use the recommendation UUID):

```
GET /api/cost-management/v1/recommendations/openshift/{recommendation_uuid}
```

Legacy aliases (identical behavior): `GET /api/cost-management/v1/recommendations/openshift/container`, `GET …/openshift/container/{recommendation_uuid}`.

The detail response includes per-term (short/medium/long) recommendations for two models:

- **Cost model**: optimizes for resource savings (p60 CPU / p95 memory)
- **Performance model**: optimizes for headroom (p98 CPU / p100 memory)

Each recommendation includes CPU/memory requests and limits, per-engine notifications on detail, and usage distribution plots under `recommendation_terms.<term>.plots.plots_data`. Each time bucket maps to `cpuUsage` and `memoryUsage` objects with percentile fields (`p50`, `p95`, `p99`, `max`) and a `format` (`cores` for CPU, `MiB` for memory). Container boxplots also include an optional `cpuThrottle` object (P95 + Max in cores); omitted when no throttling occurred. Values are assembled from daily digests (not raw samples).

Notifications: Detail responses emit `notifications` maps only on `recommendation_engines.<engine>` (not at term or top-level). List rows expose `recommendations.notification_codes` (deduplicated integer array); resolve badge text via `GET …/notification-codes`.

Example plot bucket:

```
{
  "cpuUsage": { "p50": 0.12, "p95": 0.45, "p99": 0.67, "max": 0.89, "format": "cores" },
  "cpuThrottle": { "p95": 0.03, "max": 0.12, "format": "cores" },
  "memoryUsage": { "p50": 256.0, "p95": 384.0, "p99": 420.0, "max": 512.0, "format": "MiB" }
}
```

NOTE	`cpuThrottle` is only present for container recommendations (not namespace). It is omitted entirely when both P95 and Max are zero (no throttling in that bucket).

Detail responses nest both engines under each term:

```
recommendations.recommendation_terms.medium_term.recommendation_engines.cost
recommendations.recommendation_terms.medium_term.recommendation_engines.performance
```

How engines differ (cost vs performance)

- **Cost** – lower usage percentiles (e.g. P60 CPU, P95 memory) to right-size for savings

- **Performance** – higher percentiles (e.g. P98 CPU, max memory) to preserve headroom

On list endpoints, `filter[engine]=cost` or `filter[engine]=performance` returns only the selected engine under each term and omits the other from `recommendation_engines`. When the filter is omitted, both engines are returned.

To generate cluster data where cost and performance sizing intentionally diverge (for validation or demos), use the NISE fixture at `nise/examples/ocp_dual_engine/` (`spike-cpu-api`, `steady-mem-worker` workloads; see the README in that directory).

Testing divergence

After ingesting `nise/examples/ocp_dual_engine/` data, container cost vs performance sizing will differ under `recommendation_terms.*.recommendation_engines` (higher CPU/memory on performance for spike-prone workloads). Use that fixture in your test cluster to verify dual-engine divergence in E2E, IQE, or Bruno manual runs; generic fixtures may return identical sizing while still exposing both engine blocks.

OOM Timeline

Per-day OOM (Out of Memory) kill counts for a container. Returns only days with at least one OOM event (sparse response). Defaults to the last 6 months.

```
GET /api/cost-management/v1/recommendations/openshift/containers/{recommendation_uuid}/oom-timeline
GET /api/cost-management/v1/recommendations/openshift/containers/{recommendation_uuid}/oom-timeline?start_date=2026-01-01&end_date=2026-06-29
```

Example curl:

```
curl -s -H "x-rh-identity: $IDENTITY" \
  'https://console.redhat.com/api/cost-management/v1/recommendations/openshift/containers/RECOMMENDATION_UUID/oom-timeline?start_date=2026-01-01' \
  | python3 -m json.tool
```

Example response:

```
{
  "meta": {
    "count": 9,
    "container_id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
    "start_date": "2026-01-01",
    "end_date": "2026-06-29"
  },
  "data": [
    {"date": "2026-03-15", "oom_kill_count": 3},
    {"date": "2026-03-22", "oom_kill_count": 1},
    {"date": "2026-04-01", "oom_kill_count": 5}
  ]
```

```
}
```

`meta.count` is the sum of all `oom_kill_count` values (total OOM events, not total days). Empty `data` array when no OOM events occurred in the range.

Filtering and sorting

Filter by cluster, project, workload, workload type, or container:

```
GET /api/cost-management/v1/recommendations/openshift/?filter[cluster]=CLUSTER_UUID
GET /api/cost-management/v1/recommendations/openshift/?filter[project]=my-namespace
GET /api/cost-management/v1/recommendations/openshift/?filter[namespace]=my-namespace
GET /api/cost-management/v1/recommendations/openshift/?filter[workload]=my-deployment
GET /api/cost-management/v1/recommendations/openshift/?filter[workload_type]=deployment
GET /api/cost-management/v1/recommendations/openshift/?filter[container]=my-container
```

`filter[namespace]` is an alias for `filter[project]` (both filter by OpenShift namespace). Legacy flat forms (`cluster_uuid`, `project`, `namespace`, `container`) are also accepted.

`filter[workload_type]` accepts any lowercase Kubernetes workload kind (exact match; comma-separated values are ORed), e.g. `deployment`, `statefulset`, `domain`. Legacy flat form `?workload_type=deployment` is also accepted. Invalid formats return 400; any syntactically valid kind string is accepted.

Filter by recommendation engine (limits nested `recommendation_engines` to cost or performance; omits the non-selected engine from each item):

```
GET /api/cost-management/v1/recommendations/openshift/?filter[engine]=cost
GET /api/cost-management/v1/recommendations/openshift/?filter[engine]=performance
```

Legacy flat `?engine=cost` is also accepted on container, namespace, node, VM, quality, and history endpoints.

Filter by recommendation term

Limit results to a specific recommendation term. When omitted, all terms are returned.

```
GET /api/cost-management/v1/recommendations/openshift/?filter[term]=medium_term
```

Valid values: `short_term`, `medium_term`, `long_term`

Legacy flat parameter `?term=short` is also accepted. The response `recommendation_terms` object will only contain the selected term.

Filter by GPU:

```
GET /api/cost-management/v1/recommendations/openshift/?filter[has_gpu]=true
GET /api/cost-management/v1/recommendations/openshift/?filter[gpu_model]=A100
GET /api/cost-management/v1/recommendations/openshift/?filter[gpu_classification]=underutilized
```

Filter by idle state (`active`, `idle`, or `zombie`; comma-separated values are ORed):

> **NOTE** The API uses `idle_state=zombie` which the UI displays as "Abandoned".

```
GET /api/cost-management/v1/recommendations/openshift/?filter[idle_state]=zombie,idle
```

Filter by stale data (`false` = fresh only (default), `true` = include stale and fresh, `only` = stale rows only). A recommendation is stale when the cluster has not reported within `ROS_STALENESS_THRESHOLD_HOURS` (default 48). Legacy `?stale=` accepted:

```
GET /api/cost-management/v1/recommendations/openshift/?filter[stale]=false
GET /api/cost-management/v1/recommendations/openshift/?filter[stale]=only
```

Exact-match and exclude filters are supported on `cluster`, `project`, `workload`, and `container`:

```
GET /api/cost-management/v1/recommendations/openshift/?filter[exact:cluster]=EXACT_UUID
GET /api/cost-management/v1/recommendations/openshift/?exclude[project]=openshift-monitoring
```

Filter by OpenShift labels (requires `ROS_TAGS_ENABLED=true`; tags synced from Koku):

```
GET /api/cost-management/v1/recommendations/openshift/?filter[tag:environment]=production
GET /api/cost-management/v1/recommendations/openshift/?filter[tag:environment]=production,staging
GET /api/cost-management/v1/recommendations/openshift/?filter[tag:team]=platform
```

Tag filter syntax follows Koku report conventions:

- `filter[tag:<key>]=<value>` – exact value match
- `filter[tag:<key>]=a,b` – OR within the same key (comma-separated)
- Multiple `filter[tag:*]` keys – AND across keys
- `filter[tag:<key>]=*` – key present with any value
- Legacy flat form `?tag=<key>:<value>` is also accepted

> **NOTE** `filter[tag:key]` is supported on list endpoints and fleet history (`GET …/history`). It is NOT supported on per-resource detail endpoints (`GET …/containers/{id}`, etc.).

Tag sync configuration

- `ROS_TAGS_ENABLED` (default: `true`) – master switch for tag filters and tag `group_by` on savings summary
- `ROS_TAGS_SOURCE` (default: `api` on cost-onprem chart and SaaS; `db` optional on-prem) – `api` uses Koku push sync into ROS `resolved_tags`; `db` reads Koku PostgreSQL tag tables at query time (requires shared DB access)
- Tag keys must be enabled in Cost Management **Settings ▌ Tags**; discover keys via `GET /api/cost-management/v1/tags/openshift/`

For the complete machine-readable parameter list, see `openapi.json`.

When a tag key is unknown or has no matching containers, `meta.count` is `0` and `meta.warnings` may describe the empty result.

When `filter[tag:key]=value` is combined with RBAC-scoped identity, results are the intersection of allowed clusters/namespaces AND tag matches. A tag filter scoped to a cluster outside RBAC scope returns HTTP 200 with an empty list (not 403).

Hosted control-plane recommendations

Dedicated surface for HyperShift control-plane rightsizing (requires the `hcp` plugin). Lists HCP-namespaced container rows only. HCP rows carry cluster facts from the `clusters` row: `hosted_cluster_id` when associated, `incomplete: true` only for evidenced-but-unassociated hosted rows (usage data present, but the cluster is missing from the `clusters` table), and explicit `management: true|false` from `clusters.cluster_topology` under `filter[scope]=management` (rows without the scope param are hosted rows and carry `management: false`). A management row is never `incomplete`; UIs should show the management cluster name (`cluster_alias`), not "Incomplete", for those rows:

```
GET /api/cost-management/v1/recommendations/openshift/hcp
GET /api/cost-
management/v1/recommendations/openshift/hcp?filter[hosted_cluster_id]=HOSTED_CLUSTER_ID
GET /api/cost-management/v1/recommendations/openshift/hcp?group_by[hosted_cluster_id]=*
GET /api/cost-management/v1/recommendations/openshift/hcp?filter[scope]=management
GET /api/cost-
management/v1/recommendations/openshift/hcp?filter[scope]=management&group_by[namespace]=*
GET /api/cost-management/v1/recommendations/openshift/hcp/:recommendation-id
GET /api/cost-management/v1/recommendations/openshift/hcp/:recommendation-
id?include=explanation&filter[term]=short_term&filter[engine]=cost
```

Unknown hosted IDs return HTTP 200 with an empty list (not 404); non-HCP detail IDs return 404. The HCP detail resolves against the native recommendation first, with the same shape as the plain container detail endpoint: the typed `expl_*` explanation block is folded in with `include=explanation` (the only recognized include token; unknown values are ignored), projection rows are served by default and `filter[term]` / `filter[engine]` select which projections are pinned. Like tag filters, the hosted filter applies after RBAC scoping and can never grant access outside the caller's scope. Group-by rows carry per-hosted counts (associated rows only) plus `estimated_savings` (`{"value", "units"}` summed over pinned short/cost rows, converted to display currency); `meta.currency` names the currency.

`filter[scope]=management` switches the surface to management-overhead rows: container rows on `cluster_topology = 'management'` clusters whose namespace is not a hosted control plane (not in the cluster's `hcp_namespaces`; a NULL list reads as empty). It combines with `group_by[namespace]` and any other filters. Without the param the served shape is unchanged; the scope is keyword-selectable — unknown values are rejected with HTTP 400 (`{"status": "error", "message": …}`), never ignored. Under the alternated scope, detail IDs are validated against management scope: a management row 404s without the param, a hosted row 404s with it.

Hosted control-plane fleet capacity (W4 advisory)

Fleet-capacity advisory for management clusters (requires the `hcp` plugin): how many additional hosted control planes the eligible worker set could host, per the native published-sizing model (`native_model_version`), with optional MCE reference gauges. Two endpoints:

```
GET /api/cost-management/v1/recommendations/openshift/hcp/fleet-capacity
GET /api/cost-management/v1/recommendations/openshift/hcp/fleet-
capacity?filter[cluster_uuid]=CLUSTER_UUID
GET /api/cost-management/v1/recommendations/openshift/hcp/fleet-
capacity/history?filter[cluster_uuid]=CLUSTER_UUID&limit=20&offset=0
```

Without a cluster filter the latest endpoint returns exactly one row per management cluster authorized for the caller (the discovery surface); `filter[cluster_uuid]` narrows to that cluster. `filter[cluster]` and the bare `cluster_uuid` query param resolve identically. History is a per-cluster trend: the single-cluster filter is required (missing filter is HTTP 400), rows are newest-first, and `limit`/`offset` paginate with `meta.count` as the total.

Each row carries the five scenarios (`request`, `low_qps`, `average_qps`, `medium_qps`, `high_qps`) with serve-time headroom: `null` when occupancy, worker placement, or average QPS is unknown, and withheld under affirmed node pressure (ADR-0334: an over-committed cluster must not advertise an exact `+N`). An empty `limiting_resource` means the scenario has no estimate — render "not available", never 0; a genuine zero maximum with a limiting resource is a value. `mce_references` are the optional same-scenario MCE gauges: `null` means absent, never zero.

`warnings` are explicit booleans: `stale` (age over 2x the row's own max of `collection_interval_minutes` / `upload_cycle_minutes`), `below_version_floor` (known `ocp_version` under the documented 4.17 floor — data is still served, never suppressed), `pressured`, `pressure_unknown`, `mce_may_be_stale` (MCE gauges frozen across the freshness window while native inputs moved; latest endpoint only), and `shared_infrastructure` (compact topology: management control-plane nodes double as workers — OCP sizing-guide 3.2.5 caveat; estimates may overstate dedicated capacity, still served). Unknown or unauthorized clusters are non-disclosing: HTTP 200 with empty data, never 403/404.

Pagination

Offset pagination (default):

```
GET /api/cost-management/v1/recommendations/openshift/?limit=20&offset=0
```

Keyset pagination (stable iteration; `offset` is ignored when `after` is set):

```
GET /api/cost-management/v1/recommendations/openshift/?limit=20
# Response meta includes has_next and next_cursor when more pages exist
GET /api/cost-management/v1/recommendations/openshift/?limit=20&after=<meta.next_cursor>
```

Sorting

Allowed `order_by` values: `cluster`, `project`, `workload_type`, `workload`, `container`, `last_reported`, `cpu_request_current`, `memory_request_current`, `estimated_monthly_savings`, `estimated_monthly_waste`, `idle_state`, `idle_duration_days`, and per-term variation fields: `cpu_variation_short_cost`, `cpu_variation_short_performance`, `cpu_variation_medium_cost`, `cpu_variation_medium_performance`, `cpu_variation_long_cost`, `cpu_variation_long_performance`, `memory_variation_short_cost`, `memory_variation_short_performance`, `memory_variation_medium_cost`, `memory_variation_medium_performance`, `memory_variation_long_cost`, `memory_variation_long_performance`.

`estimated_monthly_savings` sorts by persisted dollar savings (nullable; `NULLS LAST` applies in both directions). `estimated_monthly_waste` sorts by estimated monthly waste for idle/abandoned workloads.

```
GET /api/cost-management/v1/recommendations/openshift/?order_by=last_reported&order_how=desc
GET /api/cost-
management/v1/recommendations/openshift/?order_by=cpu_variation_medium_cost&order_how=desc
```

CSV export

Export container recommendations as CSV (one row per container × term × engine):

```
GET /api/cost-management/v1/recommendations/openshift/?format=csv
GET /api/cost-
management/v1/recommendations/openshift/?format=csv&limit=100&order_by=last_reported&order_how=de
sc
```

Prefer `Accept: text/csv` for explicit content negotiation. Response `Content-Type` is `text/csv`. The export is capped at 1000 list records (up to 6000 CSV rows when all three terms and both engines are present). Columns include cluster, project, workload, container, savings (`estimated_monthly_savings`, `cpu_savings`, `memory_savings`), idle state, and per-term CPU/memory requests and limits. Hosted-cluster attribution columns (`cluster_alias`, `hosted_cluster_id`, `management`, `incomplete`) are shared by the plain container CSV and the HCP CSV (`GET …/hcp?format=csv`): `management` prints `false` for classic rows and reflects `clusters.cluster_topology` on HCP rows, while `incomplete` stays `true` only for evidenced-but-unassociated rows. Use the same filters and `order_by` parameters as the JSON list endpoint.

`cpu_savings` and `memory_savings` are the per-resource CPU and memory breakdowns of `estimated_monthly_savings`. All three are empty when cost data is unavailable (code 25).

Container notification codes

Container notification codes (catalog filter `filter[plugin]=container`):

Code	Severity	When
1	WARNING	Low confidence (proportional coverage in term window)
2	WARNING	Stale data (cluster stopped reporting)

Code	Severity	When
3	CRITICAL	OOM kill events in analysis window
5	INFO	Idle workload
6	INFO	Recommendation applied
7	INFO	New workload (< 24 hours of data)
8	WARNING	Abandoned workload
9	WARNING	Memory usage trending up
21	WARNING	HPA saturated (reserved — not emitted)
22	INFO	HPA active (reserved — not emitted)
25	INFO	No cost data — savings estimate not computed
77	INFO	Sparse data — `data_days` at or below `sparse_data_threshold` (default 2)

History and quality

Node recommendations do not expose a history or quality API. Per-node history is N/A.

Recommendation snapshot history (fleet-wide, native engine — containers):

```
GET /api/cost-management/v1/recommendations/openshift/history?limit=10
GET /api/cost-management/v1/recommendations/openshift/history?container=my-
container&filter[engine]=cost&term=short_term
GET /api/cost-
management/v1/recommendations/openshift/history?filter[tag:environment]=production&limit=10
```

History accepts `filter[engine]=cost|performance` (preferred) or legacy flat `?engine=cost`. Term filter accepts canonical forms (`short_term`, `medium_term`, `long_term`); legacy values (`short`, `medium`, `long`) are also accepted and normalized. `filter[cluster]` (and legacy `?cluster=`) match the cluster **alias**, exact match — pass the `cluster_alias` the detail/row response serves, not the cluster UUID (which returns 0 rows unless it happens to be the alias). Response includes 21 `expl_*` explanation columns (nullable, omitted when nil) providing recommendation context: data days, decay half-life, CPU/memory cost/perf percentiles, usage statistics, adaptive margins, trend slopes, OOM counts, floor flags, and idle state.

Recommendation quality metrics are available for 5 entity types, each with its own endpoint and entity-specific columns:

Container quality (stability, adoption, OOM events):

```
GET /api/cost-management/v1/recommendations/openshift/quality/containers?limit=10
GET /api/cost-
management/v1/recommendations/openshift/quality/containers?filter[engine]=cost&limit=10
GET /api/cost-
```

```
management/v1/recommendations/openshift/quality/containers?order_by=stability&order_how=desc
```

The legacy path `/quality` is a backward-compatible alias for `/quality/containers`.

PVC quality (stability, adoption, days_above_threshold):

```
GET /api/cost-management/v1/recommendations/openshift/quality/pvcs?limit=10
GET /api/cost-
management/v1/recommendations/openshift/quality/pvcs?filter[engine]=cost&order_by=days_above_thre
shold&order_how=desc
```

VM quality (stability, adoption, saturation_days):

```
GET /api/cost-management/v1/recommendations/openshift/quality/vms?limit=10
GET /api/cost-
management/v1/recommendations/openshift/quality/vms?filter[engine]=performance&order_by=saturatio
n_days&order_how=desc
```

GPU MIG quality (binary stability, profile adoption, contention_days):

```
GET /api/cost-management/v1/recommendations/openshift/quality/gpu?limit=10
GET /api/cost-
management/v1/recommendations/openshift/quality/gpu?filter[engine]=cost&order_by=contention_days&
order_how=desc
```

Snapshot quality (adoption via disappearance — no stability, no bad outcome):

```
GET /api/cost-management/v1/recommendations/openshift/quality/snapshots?limit=10
GET /api/cost-
management/v1/recommendations/openshift/quality/snapshots?order_by=adoption&order_how=desc
```

Example curl for GPU quality:

```
curl -s -H "x-rh-identity: $IDENTITY" \
  'http://localhost:8000/api/cost-
management/v1/recommendations/openshift/quality/gpu?filter[engine]=cost&limit=5' \
  | python3 -m json.tool
```

Entity-specific columns by endpoint:

- `/quality/containers` — `oom_events_after_rec` (integer)
- `/quality/pvcs` — `days_above_threshold` (integer; days usage ratio exceeded 95% of capacity)
- `/quality/vms` — `saturation_days` (integer; days CPU/memory exceeded 95% of allocated)
- `/quality/gpu` — `contention_days` (integer; days SM active exceeded profile capacity)
- `/quality/snapshots` — adoption only (no stability_pct, no bad-outcome column)

Quality defaults to the cost engine when `filter[engine]` is omitted.

Retention and lifecycle (native engine):

- `ROS_HISTORY_RETENTION_DAYS` (default **90**) — `recommendation_history`, `recommendation_quality`, and `node_gpu_timeslicing_recommendation_history` partitions
- `ROS_STALE_CLEANUP_DAYS` (default **30**) — delete stale `recommendation_sets` rows (not archived to history first)
- `ROS_STALENESS_THRESHOLD_HOURS` (default **48**) — mark recommendations stale when the cluster stops reporting

Gap prevention: history and quality writes are **non-fatal**. If analytics fail (DB error, timeout), container recommendations still persist; the processor logs `pipelineDegraded` and continues. Namespace history retries on transient Kafka errors; permanent errors skip history but keep recommendations.

Namespace history (per recommendation UUID):

```
GET /api/cost-
management/v1/recommendations/openshift/namespaces/{recommendation_uuid}/history?filter[term]=sho
rt_term&filter[engine]=cost&limit=10
```

VM history uses `ROS_VM_REC_HISTORY_RETENTION_DAYS` (default **90** days); see VM recommendation history below.

Replica counts and total savings

Container detail responses include replica information collected from `kube-state-metrics` (deployments, statefulsets, daemonsets):

- `replica_info.desired` — spec-level replica count
- `replica_info.available` — currently available/ready replicas
- `replica_info.source` — `"kube_state_metrics"` (authoritative) or `"derived"` (pod count fallback)

Total impact savings multiply per-container savings by replica count.

Namespace recommendations

Namespace-level CPU/memory recommendations (aggregated across all containers in a namespace):

```
GET /api/cost-management/v1/recommendations/openshift/namespaces
GET /api/cost-management/v1/recommendations/openshift/namespaces?limit=100&offset=0
GET /api/cost-management/v1/recommendations/openshift/namespaces?filter[cluster]=CLUSTER_UUID
GET /api/cost-management/v1/recommendations/openshift/namespaces?filter[project]=my-namespace
GET /api/cost-management/v1/recommendations/openshift/namespaces?filter[namespace]=my-namespace
GET /api/cost-management/v1/recommendations/openshift/namespaces?filter[idle_state]=idle,zombie
GET /api/cost-management/v1/recommendations/openshift/namespaces?filter[stale]=false
GET /api/cost-management/v1/recommendations/openshift/namespaces?filter[stale]=true
GET /api/cost-management/v1/recommendations/openshift/namespaces?filter[stale]=only
GET /api/cost-management/v1/recommendations/openshift/namespaces?filter[engine]=cost
GET /api/cost-management/v1/recommendations/openshift/namespaces?filter[engine]=performance
```

```
GET /api/cost-
management/v1/recommendations/openshift/namespaces?filter[tag:environment]=production
GET /api/cost-
management/v1/recommendations/openshift/namespaces?filter[tag:environment]=production,staging&fil
ter[tag:team]=platform
GET /api/cost-
management/v1/recommendations/openshift/namespaces?order_by=cpu_variation_medium_cost&order_how=d
esc
GET /api/cost-management/v1/recommendations/openshift/namespaces?format=csv
```

Canonical list path. Legacy alias (identical behavior): `GET /api/cost-management/v1/openshift/namespace/recommendations`.

List filters:

- `filter[cluster]` — cluster UUID (include mode by default; `filter[exact:cluster]`, `exclude[cluster]` for exact/exclude). Legacy flat `cluster` / `cluster_uuid` accepted.
- `filter[project]` — namespace name (comma-separated OR). `filter[namespace]` is an alias for `filter[project]`; flat `project` / `namespace` also accepted.
- `filter[idle_state]` — `active`, `idle`, or `zombie` (comma-separated values are ORed). Aggregated from child container/GPU rows.
- `filter[stale]` — `false` (default, fresh only), `true` (include stale and fresh), or `only` (stale rows only). Stale when the cluster has not reported within `ROS_STALENESS_THRESHOLD_HOURS` (default 48). Legacy `?stale=` accepted.
- `filter[engine]` — `cost` or `performance`; omits the other engine from `recommendation_engines` on each item (legacy `?engine=` also accepted).
- `filter[term]` — `short`, `medium`, `long` or `short_term`, `medium_term`, `long_term`; limits nested `recommendation_terms` in each list item (legacy `?term=` also accepted). When omitted, list items default to `short_term` cost only.
- `filter[tag:<key>]` — tag-based filtering when `ROS_TAGS_ENABLED=true` (tags synced from Koku). Multiple `filter[tag:*]` keys are ANDed; comma-separated values within one key are ORed. See container filtering above for full syntax.

Sorting: `order_by` + `order_how` (`asc` or `desc`), or bracket `order_by[<field>]=asc|desc`.

- `cluster`, `project`, `last_reported`
- Variation columns (percent change vs current request): `cpu_variation_short_cost`, `cpu_variation_short_performance`, `cpu_variation_medium_cost`, `cpu_variation_medium_performance`, `cpu_variation_long_cost`, `cpu_variation_long_performance`, `memory_variation_short_cost`, `memory_variation_short_performance`, `memory_variation_medium_cost`, `memory_variation_medium_performance`, `memory_variation_long_cost`, `memory_variation_long_performance`

```
GET /api/cost-management/v1/recommendations/openshift/namespaces?order_by=project&order_how=asc
GET /api/cost-
management/v1/recommendations/openshift/namespaces?order_by=last_reported&order_how=desc
```

CSV export (native list): prefer `Accept: text/csv` or `?format=csv`. Same filters and `order_by` as the JSON list.

Term settings (per-tenant short/medium/long windows):

```
GET /api/cost-
management/v1/recommendations/openshift/settings/terms?recommendation_type=namespace
PUT /api/cost-
management/v1/recommendations/openshift/settings/terms?recommendation_type=namespace
DELETE /api/cost-
management/v1/recommendations/openshift/settings/terms?recommendation_type=namespace
```

Business hours: when `ROS_BUSINESS_HOURS_ENABLED=true` and a schedule applies (org, cluster, or namespace scope), detail payloads include `recommendation_engines.{cost|performance}.business_hours` after reship completes. See Business hours settings.

Notification codes on namespace rows: **1** (low confidence), **2** (stale data), **7** (new workload), **9** (memory trending up), **77** (sparse data — limited observation days). Catalog: `GET …/notification-codes?filter[plugin]=namespace`.

Savings fields on namespace responses: `recommendations.estimated_monthly_savings` (total, `{value, units}`), `recommendations.cpu_savings` (CPU portion, `{value, units}`), `recommendations.memory_savings` (memory portion, `{value, units}`), and top-level `estimated_monthly_waste` for idle/zombie namespaces (`{value, units}`). Requires `ROS_SAVINGS_ESTIMATES_ENABLED=true` (default) and cost model rates from Koku Masu. When cost data is unavailable, savings fields are omitted.

Detail for a specific namespace recommendation:

```
GET /api/cost-management/v1/recommendations/openshift/namespaces/{recommendation_uuid}
```

History for a namespace recommendation (`filter[term]` and `filter[engine]` apply here only):

```
GET /api/cost-
management/v1/recommendations/openshift/namespaces/{recommendation_uuid}/history?filter[term]=sho
rt_term&filter[engine]=cost&limit=10
GET /api/cost-
management/v1/recommendations/openshift/namespaces/{recommendation_uuid}/history?filter[term]=med
ium_term&filter[engine]=performance&limit=30
```

History `filter[term]` values: `short_term`, `medium_term`, `long_term`. `filter[engine]`: `cost` or `performance`.

GPU recommendations hub (summary)

Lightweight counts plus absolute links to the time-slicing and MIG listing endpoints:

```
GET /api/cost-management/v1/recommendations/openshift/gpu
```

NVIDIA MIG (Multi-Instance GPU) recommendations

MIG bin-packing recommendations appear in the container detail response and in a dedicated list endpoint.

Detail response — when a container uses a MIG-capable GPU (A100, H100, H200, B200), the response includes a `gpu` map keyed by term (`short`, `medium`, `long`). Each term object may include:

- `current_gpu_model` / `current_gpu_profile` — what the container currently uses
- `recommended_gpu_profile` — smallest MIG profile whose resources cover P98 frame-buffer usage (with headroom)
- `gpu_classification` — e.g. `compute_bound_underutil`, `memory_bound`, `underutilized`
- `estimated_monthly_gpu_savings` — structured savings object (`value` + `units`, typically USD)
- `notifications` — advisory codes **10**, **26**–**28** on the term object

```
GET /api/cost-management/v1/recommendations/openshift/{recommendation_uuid}
```

List containers that have a non-full-GPU MIG recommendation:

```
GET /api/cost-management/v1/recommendations/openshift/gpu/mig
GET /api/cost-management/v1/recommendations/openshift/gpu/mig?limit=20&offset=0
GET /api/cost-management/v1/recommendations/openshift/gpu/mig?limit=20&after=<meta.next_cursor>
GET /api/cost-
management/v1/recommendations/openshift/gpu/mig?filter[cluster]=<cluster_uuid>&filter[project]=ml
-team
GET /api/cost-management/v1/recommendations/openshift/gpu/mig?filter[tag:gpu_pool]=high-memory
GET /api/cost-management/v1/recommendations/openshift/gpu/mig?order_by=confidence&order_how=desc
GET /api/cost-management/v1/recommendations/openshift/gpu/mig?format=csv
```

Test data: Generate OCP payloads with GPU ROS metrics using `nise report ocp --ros-ocp-info` (plus a GPU-enabled static YAML). Package typed CSVs with `manifest.json` and upload via ingress; see ros-ocp-backend `docs/features/gpu-mig.md` for MIG fixture columns (`gpu_model`, `gpu_uuid`, `instance_name`, `utilization`).

Returns container-term rows with `recommended_gpu_profile` (MIG slice, not `full_gpu`). Uses recommendation **terms** (`short`/`medium`/`long`), not `filter[engine]=cost|performance`. Each row includes `id` (container recommendation id for `GET …/recommendations/openshift/{id}`) and `workload_type`. Duplicate `id` values across term (and GPU-model) rows for the same container are expected. Group-by rows omit `id`. CSV adds trailing `id` and `workload_type` columns.

Plugin gating: GPU MIG routes are served by the `gpu` recommendation plugin. Use deployment env vars `ROS_ENABLED_PLUGINS` (allowlist) or `ROS_DISABLED_PLUGINS` (denylist when allowlist is empty) to enable or disable plugins; omit `gpu` from the enabled set (or add `gpu` to the disabled set) to turn off GPU recommendations. When the `gpu` plugin is disabled, `GET …/recommendations/openshift/gpu/mig` and related `/gpu/ paths return *404` (`plugin 'gpu' is not enabled`), not an empty list. `GET …/settings/gpu` follows the same gating. `order_by` values: `cluster_uuid`, `namespace`, `workload`, `container`, `term`, `gpu_model`, `confidence`. `estimated_monthly_gpu_savings` (structured object) is on container detail (`GET …/recommendations/openshift/{uuid}` ▌ `gpu.{term}`), not this list. MIG list CSV uses column `gpu_classification` (matches JSON). Supports pagination, `format=csv`, and filters per

OpenAPI (`filter[cluster]`, `filter[project]`, `filter[gpu_idle_state]`, tag filters).

GPU classification thresholds (MIG sizing, idle/underutilized detection):

```
GET /api/cost-management/v1/recommendations/openshift/settings/gpu
GET /api/cost-management/v1/recommendations/openshift/settings/thresholds?recommendation_type=gpu
```

Returns fields such as `idle_threshold`, `underutilized_sm_threshold`, `fb_headroom_factor`, and `mig_fb_percentile`. Term windows: `GET …/settings/terms?recommendation_type=gpu`.

RBAC: Same `openshift.cluster` permission as other ROS GPU routes; node-scoped rows also require `openshift.node`. A `filter[cluster]` for a cluster outside the caller's scope returns an empty list (HTTP 200), not 403.

Notifications: MIG right-sizing and GPU idle guidance surface on **container detail** (`GET …/recommendations/openshift/{recommendation_uuid}`) in the `notifications` array (codes **10**, **26–28**). The MIG list endpoint does not include savings or notifications.

NVIDIA GPU time-slicing recommendations

Node-level GPU time-slicing consolidation guidance. Evaluates how many containers could share each physical GPU via `nvidia.com/gpu.replicas`. Uses recommendation **terms** (`short`/`medium`/`long`), not the cost/performance dual engine (`filter[engine]=cost|performance` applies to container, namespace, and node CPU/memory routes only).

Recommendations are **computed and persisted at ingest** into `node_gpu_timeslicing_recommendations` (replicas, confidence, savings as `estimated_savings_cents`). The list endpoint reads from this table when the org has any persisted rows. Orgs without persisted data use a **compute-at-read fallback** (digest scan + engine) until the next ingest cycle or [Backfill GPU time-slicing].

```
GET /api/cost-management/v1/recommendations/openshift/gpu/timeslicing
GET /api/cost-management/v1/recommendations/openshift/gpu/timeslicing?limit=25&offset=0
GET /api/cost-
management/v1/recommendations/openshift/gpu/timeslicing?limit=25&after=<meta.next_cursor>
GET /api/cost-
management/v1/recommendations/openshift/gpu/timeslicing?filter[cluster]=<cluster_uuid>
GET /api/cost-management/v1/recommendations/openshift/gpu/timeslicing?filter[gpu_model]=T4
GET /api/cost-
management/v1/recommendations/openshift/gpu/timeslicing?filter[tag:environment]=production
GET /api/cost-management/v1/recommendations/openshift/gpu/timeslicing?node_name=my-gpu-
node&gpu_model=T4
GET /api/cost-
management/v1/recommendations/openshift/gpu/timeslicing?order_by=confidence&order_how=desc
GET /api/cost-management/v1/recommendations/openshift/gpu/timeslicing?format=csv
```

Returns per-node recommendations with:

- `gpu_model` – NVIDIA GPU model on the node

- `recommended_replicas` – suggested time-slicing replica count
- `savings_per_gpu` / `total_node_savings` – estimated cost savings (`{value, units}` [`MoneyAmount`])
- `notification_codes` – includes **36** when time-slicing is recommended

`order_by` values: when persisted data exists – `node_name` (default), `cluster_uuid`, `gpu_model`, `recommended_replicas`, `confidence`, `total_node_savings`. On the compute-at-read fallback path, only `node_name`, `cluster_uuid`, and `gpu_model` are supported; `recommended_replicas`, `confidence`, and `total_node_savings` return HTTP 400 (post-aggregate fields that cannot be sorted at scale). Supports pagination (`limit`, `offset`) and `format=csv` (or `Accept: text/csv`). No `filter[project]` – node-level scope only.

Plugin gating: Same `gpu` plugin as MIG (`ROS_ENABLED_PLUGINS` / `ROS_DISABLED_PLUGINS`).

RBAC: Same `openshift.cluster` permission as other ROS GPU routes; node-scoped rows also require `openshift.node`. A `filter[cluster]` for a cluster outside the caller's scope returns an empty list (HTTP 200), not 403.

Settings: Time-slicing thresholds on `GET …/settings/gpu` include `timeslicing_min_replicas`, `timeslicing_max_replicas`, `timeslicing_majority_threshold`, `timeslicing_base_penalty`, and `timeslicing_impacted_weight`. Term windows: `GET …/settings/terms?recommendation_type=gpu`.

Cold-start signal: `meta.data_days_available` (integer) reports distinct days of `daily_node_digests` data for the queried cluster(s). Compare against `min_data_days` from `GET …/settings/terms?recommendation_type=gpu` to distinguish cold-start (insufficient data) from genuine "no recommendations".

GPU time-slicing detail

Peak hours replica sizing lives on the **detail** route, not the list:

```
GET /api/cost-management/v1/recommendations/openshift/gpu/timeslicing/{node}
GET /api/cost-
management/v1/recommendations/openshift/gpu/timeslicing/{node}?cluster_uuid=<cluster_uuid>
GET /api/cost-
management/v1/recommendations/openshift/gpu/timeslicing/{node}?cluster_uuid=<cluster_uuid>&filter
[gpu_model]=T4&filter[term]=medium
```

Requires an **org ▌cluster** business-hours schedule (namespace-only schedules omit the nest). When nested `business_hours.recommended_replicas` is present, `business_hours.notifications` includes code **81** (`GPU_TS_BH_CLUSTER_WINDOW`) – overnight training and off-hours bursts are excluded. Parent `notification_codes` stay all-hours (**36**). Code **81** is not on the list, history, GPU summary `timeslicing.count`, or parent codes. Reason-only insufficient-data nests omit 81.

GPU time-slicing history

Append-only snapshots of how time-slicing recommendations evolved for a node × GPU model × term:

```
GET /api/cost-management/v1/recommendations/openshift/gpu/timeslicing/history
GET /api/cost-
management/v1/recommendations/openshift/gpu/timeslicing/history?cluster_uuid=<cluster_uuid>&node_
```

```
name=<node_name>
GET /api/cost-
management/v1/recommendations/openshift/gpu/timeslicing/history?cluster_uuid=<cluster_uuid>&node_
name=<node_name>&gpu_model=T4&term=medium&limit=20&offset=0&order_by=recorded_at&order_how=desc
```

Required query params: `cluster_uuid`, `node_name`. Optional: `gpu_model`, `term`, `limit` (default 20), `offset`, `order_by` (`recorded_at` default, or `recommended_replicas`, `confidence`, `candidate_count`, `impacted_count`), `order_how` (`asc` | `desc`).

Returns paginated rows from `node_gpu_timeslicing_recommendation_history` with `recommended_replicas`, `confidence`, `candidate_count`, `impacted_count`, `estimated_savings_cents`, and `recorded_at`. Retention: **90 days** (`ROS_HISTORY_RETENTION_DAYS`, default 90).

RBAC: Same `openshift.cluster` and `openshift.node` permissions as the time-slicing list. Unknown or out-of-scope `cluster_uuid` returns an empty list (HTTP 200).

Node CPU/memory utilization recommendations

Node-level right-sizing based on CPU and memory utilization patterns — identifies underutilized, overcommitted, stranded-resource, idle, and zombie nodes. One JSON object per node with nested `recommendation_terms` and dual engines (`cost`, `performance`).

List stays all-hours. Node **detail** may nest `recommendation_engines.{cost|performance}.business_hours` (cores/GiB) when an org or cluster schedule is enabled (code **79**). Visual Insights may include sibling `daily_digests_business_hours` (same shape as `daily_digests`; omitted when empty; never merged). Namespace-only schedules do not apply to nodes.

```
GET /api/cost-management/v1/recommendations/openshift/nodes
GET /api/cost-management/v1/recommendations/openshift/nodes?limit=10&offset=0
GET /api/cost-management/v1/recommendations/openshift/nodes?limit=10&after=<meta.next_cursor>
GET /api/cost-
management/v1/recommendations/openshift/nodes?filter[term]=medium_term&filter[engine]=cost
GET /api/cost-
management/v1/recommendations/openshift/nodes?filter[cluster]=CLUSTER_UUID&filter[node]=worker-
0&limit=1
GET /api/cost-management/v1/recommendations/openshift/nodes?format=csv
```

List filters (bracket syntax preferred): `filter[cluster]`, `filter[node]`, `filter[term]` (`short_term`|`medium_term`|`long_term`; legacy `short`|`medium`|`long` accepted), `filter[engine]` (`cost`|`performance`), `filter[is_underutilized]`, `filter[is_overcommitted]`, `filter[idle_state]` (comma-separated `active`, `idle`, `zombie`), `filter[stranded_resource]` (`cpu`, `memory`, `none`), `filter[instance_type]`, `filter[machineset_name]`, `filter[tag:<key>]=<value>` (requires `ROS_TAGS_ENABLED=true`; nodes match when any workload namespace on the cluster has the label). Sort: `order_by` (`estimated_monthly_savings` default, or `node`), `order_how` (`asc`|`desc`). Pagination: `limit`, `offset`. Legacy flat params `?term=`, `?engine=`, `?cluster_uuid=` still work.

RBAC: node list and detail require `cost-management.openshift.node` permission (node name scope) in addition to cluster access via `cost-management.openshift.cluster`. Empty results when the user has no intersecting cluster/node permissions.

Single-node detail: `GET …/nodes/{node}?filter[cluster]=CLUSTER_UUID` (registered). List-filter equivalent: `filter[node]` + `filter[cluster]` with `limit=1` (same response shape).

```
GET /api/cost-management/v1/recommendations/openshift/nodes?filter[is_underutilized]=true
GET /api/cost-management/v1/recommendations/openshift/nodes?filter[stranded_resource]=memory
GET /api/cost-management/v1/recommendations/openshift/nodes?filter[machineset_name]=worker
GET /api/cost-management/v1/recommendations/openshift/nodes?filter[tag:environment]=production
```

Response fields (per node): `classification` (including `idle_state`, `stranded_resource`), `metrics` (CPU/memory P50/P95), `pod_count`, `pod_capacity`, `pod_scheduling_headroom`, `instance_type`, `machineset_name`, `suggested_instance_type`, `instance_type_reason`, `cpu_overcommit_ratio`, `trend_slope`, nested `recommendation_terms.<term>.recommendation_engines.{cost,performance}` with sizing, `node_count_reduction`, `estimated_monthly_savings` (`value` + `units`), and per-engine `notifications` (map keyed by code string). Node notification codes: **1** low confidence, **11** underutilized, **12** overcommitted, **13** stranded resources (`suggested_direction` when CPU/memory imbalanced), **15** idle/zombie, **25** no cost data, **74** pod scheduling limit, **76** fleet consolidation (message includes MachineSet when applicable), **77** sparse data (limited observation days). Detail (`GET …/nodes/{node}`) also returns top-level aggregated `notifications`. No top-level `notification_codes` array on list rows.

`GET /api/cost-management/v1/recommendations/openshift/notification-codes?filter[plugin]=node` returns the node plugin catalog (emitted + reserved codes).

CSV export (`?format=csv` or `Accept: text/csv`): one row per node × term × engine; columns include `pod_capacity`, `pod_scheduling_headroom`, classification, and savings.

Fleet consolidation groups underutilized nodes by `instance_type` when the operator provides it; otherwise by similar allocatable CPU/memory capacity.

MachineSet aggregation recommendations

Groups per-node recommendations by OpenShift MachineSet name (Tier 2 list API; aggregation only — no separate engine table):

```
GET /api/cost-management/v1/recommendations/openshift/machinesets
GET /api/cost-
management/v1/recommendations/openshift/machinesets?filter[term]=medium&limit=10&offset=0
GET /api/cost-
management/v1/recommendations/openshift/machinesets?limit=10&after=<meta.next_cursor>
GET /api/cost-management/v1/recommendations/openshift/machinesets?filter[cluster]=CLUSTER_UUID
GET /api/cost-management/v1/recommendations/openshift/machinesets?filter[machineset_name]=worker*
```

Filters: `filter[cluster]` / `cluster_uuid`, `filter[machineset_name]` (exact or `*` wildcard), `filter[term]` / `term` (`short`|`medium`|`long`, default `medium`). Sorted by total monthly savings descending (cost engine rows only). Pagination: `limit`, `offset` (default 0), or keyset via `after` from `meta.next_cursor` (ignores `offset` when set).

CSV export (`?format=csv` or `Accept: text/csv`): one row per MachineSet group; savings columns are `total_monthly_savings_value` and `total_monthly_savings_units`.

Response fields (per MachineSet): `machineset_name`, `cluster_uuid`, `cluster_alias`, `instance_type`, `current_node_count`, `recommended_node_count`, `excess_nodes`, `total_monthly_savings` (`value` + `units` [`MoneyAmount`]), `avg_cpu_utilization`, `avg_memory_utilization`, `nodes`. List `meta.currency` is the ISO 4217 code for savings (default `USD`). No `confidence_level` on MachineSet rows (aggregation only; confidence applies on per-node list/detail).

MachineAutoscaler integration is planned as a future enhancement (Tier 3).

Deprecated path (backward compatibility)

Historically, `/recommendations/openshift/nodes` served GPU time-slicing listings; that behavior now lives under `/recommendations/openshift/gpu/timeslicing`. The old CPU/memory listing path was `/recommendations/openshift/nodes/utilization`.

```
GET /api/cost-management/v1/recommendations/openshift/nodes/utilization
```

Deprecated alias for `GET /api/cost-management/v1/recommendations/openshift/nodes`. Prefer the canonical `/nodes` URL. Responses include `Deprecation: true` and a `Link` header to `/nodes`. IQE and new integrations should call `/nodes` only.

Node hourly utilization

Hourly CPU and memory utilization digests for a single node, enabling heatmap and time-of-day charts. Requires Visual Insights enabled (`ROS_VISUAL_INSIGHTS_ENABLED=true`) and hourly node digests enabled (`ROS_HOURLY_NODE_DIGESTS_ENABLED=true`).

```
GET /api/cost-management/v1/recommendations/openshift/node/{node-name}/hourly-
utilization?cluster_uuid=CLUSTER_UUID
GET /api/cost-management/v1/recommendations/openshift/node/{node-name}/hourly-
utilization?cluster_uuid=CLUSTER_UUID&days=30
```

Required: path parameter `node-name`, query parameter `cluster_uuid` (or `filter[cluster]`). Optional: `days` (default 14, max 90).

Each data point returns `report_date`, `hour` (0–23), `cpu_usage_p95_mc`, `mem_usage_p95_kib`, `sample_count`, `max_pod_count`. Data is stored in `hourly_node_digests`; retention controlled by `ROS_HOURLY_NODE_DIGESTS_RETENTION_DAYS` (default 90).

PVC (Persistent Volume Claim) recommendations

Right-sizing recommendations for persistent volumes — oversized, near-full, orphaned, healthy, and growth trend projection. List returns one row per PVC for the selected **term** (default `medium`). Use the detail endpoint for all terms plus daily usage history.

```
GET /api/cost-management/v1/recommendations/openshift/pvcs
GET /api/cost-management/v1/recommendations/openshift/pvcs?limit=20&offset=0
GET /api/cost-management/v1/recommendations/openshift/pvcs?limit=20&after=<meta.next_cursor>
```

```
GET /api/cost-
management/v1/recommendations/openshift/pvcs?filter[term]=medium&filter[recommendation_type]=over
sized
GET /api/cost-
management/v1/recommendations/openshift/pvcs?filter[cluster]=CLUSTER_UUID&filter[project]=product
ion
GET /api/cost-management/v1/recommendations/openshift/pvcs?filter[storageclass]=gp3-csi
GET /api/cost-management/v1/recommendations/openshift/pvcs?filter[tag:environment]=production
GET /api/cost-
management/v1/recommendations/openshift/pvcs?order_by=estimated_monthly_savings&order_how=desc
GET /api/cost-management/v1/recommendations/openshift/pvcs?format=csv
```

Detail for a single PVC (all configured terms + `historical_usage`):

```
GET /api/cost-
management/v1/recommendations/openshift/pvcs/detail?cluster_uuid=CLUSTER_UUID&namespace=productio
n&persistentvolumeclaim=old-logs
GET /api/cost-
management/v1/recommendations/openshift/pvcs/detail?filter[cluster]=CLUSTER_UUID&filter[project]=
production&persistentvolumeclaim=old-logs
```

PVC threshold and term settings:

```
GET /api/cost-management/v1/recommendations/openshift/settings/pvc
PUT /api/cost-management/v1/recommendations/openshift/settings/pvc
DELETE /api/cost-management/v1/recommendations/openshift/settings/pvc
GET /api/cost-management/v1/recommendations/openshift/settings/terms?recommendation_type=pvc
```

Available list filters:

- `filter[cluster]` — cluster UUID (legacy `cluster_uuid`, `cluster`)
- `filter[project]` — namespace (legacy `namespace`, `project`)
- `filter[recommendation_type]` — `oversized`, `near_full`, `orphaned`, or `healthy` (legacy `recommendation_type`)
- `filter[term]` — `short`, `medium`, or `long` (default `medium`); aliases `short_term`, `medium_term`, `long_term` also accepted
- `filter[storageclass]` — StorageClass name (exact match)
- `filter[tag:<key>]` — OpenShift label tag (requires `ROS_TAGS_ENABLED=true`)
- `order_by` — `usage_ratio` (default), `estimated_monthly_savings`, `pvc_name` / `persistentvolumeclaim`, `capacity_bytes`
- `order_how` — `asc` or `desc` (default `desc`)
- `limit` (1–100, default 20), `offset` (default 0)
- `after` — keyset cursor from `meta.next_cursor` (ignores `offset` when set); see pagination docs
- `format=csv` or `Accept: text/csv` — flattened CSV export (same filters as JSON list; includes `confidence_level` column)

Example request:

```
curl -s -H "x-rh-identity: ${IDENTITY}" \
  'http://localhost:8000/api/cost-
management/v1/recommendations/openshift/pvcs?filter[recommendation_type]=orphaned&filter[term]=me
dium&limit=10'
```

Example list response snippet:

```
{
  "meta": { "count": 1, "limit": 10, "offset": 0, "has_next": false, "currency": "USD" },
  "data": [
    {
      "cluster_uuid": "550e8400-e29b-41d4-a716-446655440001",
      "namespace": "production",
      "persistentvolumeclaim": "old-logs",
      "mounted_by": "virt-launcher-old-logs-x9y8z",
      "vm_name": "fedora-vm",
      "storageclass": "gp3",
      "capacity_bytes": 107374182400,
      "usage_ratio": 0.0,
      "recommendation_type": "orphaned",
      "confidence_level": 1.0,
      "idle_since": "2026-05-01",
      "idle_duration_days": 14,
      "days_to_full": null,
      "growth_bytes_per_day": 0,
      "estimated_monthly_savings": { "value": "12.50", "units": "USD" },
      "term": "medium",
      "notifications": {
        "20": { "type": "WARNING", "message": "PVC has zero usage...", "code": 20 }
      },
      "resize_note": "This PVC has zero usage. If the data is no longer needed, deleting the PVC
will reclaim the backing storage volume."
    }
  ]
}
```

`estimated_monthly_savings` uses structured `{value, units}` (not a flat USD float). Savings are computed for **oversized** (capacity reduction) and **orphaned** (full provisioned capacity reclaim) when `KOKU_MASU_URL` and `ROS_SAVINGS_ESTIMATES_ENABLED=true`. Code **25** is appended when cost data is unavailable.

PVC notification codes (catalog filter `filter[plugin]=pvc`):

Code	Severity	When
1	WARNING	Low confidence (proportional coverage in term window)
20	WARNING	Zero usage across all intervals (orphaned)
25	INFO	No cost data — savings estimate not computed
29	INFO	Capacity significantly exceeds sustained usage (oversized)

Code	Severity	When
30	WARNING	Usage approaching capacity or growth trend (near-full)
77	INFO	Sparse data — `data_days` at or below `sparse_data_threshold` (default 2)

```
GET /api/cost-management/v1/recommendations/openshift/notification-codes?filter[plugin]=pvc
```

ResourceQuota recommendations

Namespace-level Kubernetes `ResourceQuota` right-sizing (distinct from namespace CPU/memory usage recommendations). Compares configured quota hard limits and used consumption against aggregated container recommendation totals.

Quota savings (`estimated_savings`) are per-recommendation only. They are NOT included in the fleet `GET /savings-summary` rollup (which covers container, node, PVC, snapshot, and VM plugins).

ResourceQuota recommendations are not engine- or term-selectable: the quota engine uses fixed `term=medium` and `engine=cost` container aggregates (no `filter[term]` on this API).

When RBAC is enabled, list results are scoped to clusters the caller may read (`openshift.cluster`). `filter[cluster]` for an unknown or unauthorized cluster returns **200** with an empty list.

The first ingestion cycle after enabling quota collects namespace quota snapshots; recommendations that depend on container aggregates may appear from the **second** cycle onward when container CSV is present in the payload.

Object-count metrics (`objectcounts. / aggregated object_count_` in operator CSVs) are ingested internally but are **not** exposed in API `quota_hard` / `quota_used` / `utilization` fields. High object-count utilization still drives `raise` / `high` `risk_level` and notification code **72** (`QUOTA_BLOCKING`); object-count capacity is not monetized in `estimated_savings`.

```
GET /api/cost-management/v1/recommendations/openshift/quota/
GET /api/cost-management/v1/recommendations/openshift/quota/?format=csv&limit=100
GET /api/cost-management/v1/recommendations/openshift/quota/?filter[cluster]=550e8400-e29b-41d4-
a716-446655440000
GET /api/cost-management/v1/recommendations/openshift/quota/?filter[project]=production
GET /api/cost-management/v1/recommendations/openshift/quota/?filter[tag:environment]=production
GET /api/cost-management/v1/recommendations/openshift/quota/?filter[quota_name]=compute-quota
GET /api/cost-
management/v1/recommendations/openshift/quota/?filter[recommendation_type]=tighten&filter[risk_le
vel]=low
GET /api/cost-
management/v1/recommendations/openshift/quota/?order_by=utilization&order_how=desc&limit=20&offse
t=0
GET /api/cost-
management/v1/recommendations/openshift/quota/?group_by[cluster]=*&limit=20&offset=0
GET /api/cost-
management/v1/recommendations/openshift/quota/?group_by[project]=*&limit=20&offset=0
```

```
GET /api/cost-management/v1/recommendations/openshift/quota/detail?cluster_uuid=550e8400-e29b-41d4-a716-446655440001&namespace=production&quota_name=compute-quota
```

Available filters:

- `filter[cluster]` — cluster UUID (exact match)
- `filter[project]` — namespace name (exact match)
- `filter[tag:<key>]` — namespace tag (requires `ROS_TAGS_ENABLED=true`)
- `filter[quota_name]` — ResourceQuota object name (alias `filter[resource_quota_name]`)
- `filter[recommendation_type]` — `tighten`, `raise`, `optimal`, or `none` (no recommendation generated yet)
- `filter[risk_level]` — `high`, `medium`, `low`, or `none`
- `group_by[cluster]` or `group_by[project]` — aggregate rows (mutually exclusive)
- `order_by` — `quota_name`, `utilization`, `risk_level`, `namespace`, or `estimated_monthly_savings`
- `order_how` — `asc` or `desc` (default `desc` for utilization)
- `limit` (1–100, default 20), `offset` (default 0)
- `after` — keyset cursor from `meta.next_cursor` (ignores `offset` when set); see pagination docs

Example request:

```
curl -s -H "x-rh-identity: ${IDENTITY}" \
  'http://localhost:8000/api/cost-management/v1/recommendations/openshift/quota/?filter[recommendation_type]=tighten&limit=10'
```

Keyset next page:

```
GET /api/cost-management/v1/recommendations/openshift/quota/?limit=20&after=<meta.next_cursor>
```

Example response snippet:

```
{
  "meta": { "count": 1, "limit": 10, "offset": 0, "currency": "USD" },
  "links": { "first": "...", "last": "..." },
  "data": [
    {
      "cluster_uuid": "550e8400-e29b-41d4-a716-446655440001",
      "namespace": "production",
      "quota_name": "compute-quota",
      "recommendation_type": "tighten",
      "risk_level": "low",
      "quota_hard": {
        "cpu_request_millicores": 100000,
        "cpu_limit_millicores": null,
        "memory_request_bytes": null,
        "memory_limit_bytes": null
      },
      "quota_used": { "cpu_request_millicores": 25000 },
```

```
      "quota_recommended": { "cpu_request_millicores": 36000 },
      "utilization": { "cpu_request_percent": 25.0 },
      "capacity_freed": { "cpu_millicores": 64000, "memory_bytes": 0 },
      "estimated_savings": { "value": "12.34", "units": "USD" },
      "last_observed_at": "2026-05-28T12:00:00Z"
    }
  ]
}
```

Notification codes

Namespace ResourceQuota rows may emit notification codes **70–72**. Filter the catalog with `GET /api/cost-management/v1/recommendations/openshift/notification-codes/?filter[plugin]=quota`. Code **73** (`CLUSTER_QUOTA_AT_CAPACITY`) applies to ClusterResourceQuota rows only.

Code	Name	Description
70	QUOTA_NEAR_CAPACITY	Quota at high utilization risk (near capacity)
71	QUOTA_OVERSIZED	Tighten recommendation — quota has significant unused headroom
72	QUOTA_BLOCKING	Quota at or over hard limit — pods may be pending

Detail and history

`GET /api/cost-management/v1/recommendations/openshift/quota/detail` returns one recommendation object (not wrapped in `data`). Required query parameters: `cluster_uuid`, `namespace` (alias `filter[project]`). Optional: `quota_name` (alias `filter[resource_quota_name]`) when multiple ResourceQuota objects exist in the namespace.

The response includes the same fields as a list row plus `headroom_basis_points` and a `history[]` array of per-resource snapshots for trend charts. Each history entry includes `recorded_at`, `resource`, `recommendation_type`, `risk_level`, and optional `recommended_hard`, `current_hard`, `current_used`, and `utilization_percent` from the previous observation.

Example detail request:

```
curl -s -H "x-rh-identity: ${IDENTITY}" \
  'http://localhost:8000/api/cost-
management/v1/recommendations/openshift/quota/detail?cluster_uuid=550e8400-e29b-41d4-a716-
446655440001&namespace=production&quota_name=compute-quota'
```

Requires the `quota` plugin (`ROS_ENABLED_PLUGINS`); when disabled the route returns `404`.

Quota recommendation settings (per-organization thresholds; env vars lock fields in `locked_fields`):

```
GET /api/cost-management/v1/recommendations/openshift/settings/quota
PUT /api/cost-management/v1/recommendations/openshift/settings/quota
DELETE /api/cost-management/v1/recommendations/openshift/settings/quota
```

Example GET/PUT body:

```
{
  "headroom_percent": 10,
  "high_risk_threshold_percent": 90,
  "medium_risk_threshold_percent": 70
}
```

PUT validation: `headroom_percent` 0–100; `high_risk_threshold_percent` 1–100 and greater than `medium_risk_threshold_percent`; `medium_risk_threshold_percent` 1–99. DELETE removes per-org overrides (falls back to `ROS_QUOTA_*` env or defaults 10/90/70).

Quota headroom trend

Per-day quota hard limit vs actual usage for CPU requests and memory requests, enabling headroom trend charts. Requires Visual Insights enabled (`ROS_VISUAL_INSIGHTS_ENABLED=true`) and the quota plugin.

```
GET /api/cost-management/v1/recommendations/openshift/quota/{quota-id}/trend
GET /api/cost-management/v1/recommendations/openshift/quota/{quota-id}/trend?start_date=2026-06-01&end_date=2026-06-30
```

Path parameter: `quota-id` (quota recommendation UUID). Optional query: `start_date` (YYYY-MM-DD, default 30 days ago), `end_date` (YYYY-MM-DD, default today).

Each data point returns `date`, `cpu_request_hard_millicores`, `cpu_request_used_millicores`, `memory_request_hard_bytes`, `memory_request_used_bytes`. Values are nullable when the metric was not collected on that date. Data is sourced from `daily_namespace_quota_digests`.

ClusterResourceQuota recommendations

OpenShift-only team/tenant **ClusterResourceQuota** right-sizing (distinct from namespace `ResourceQuota` and namespace usage recommendations). Compares CRQ hard/used metrics from `openshift_clusterresourcequota_usage` against aggregated namespace quota recommendation totals.

```
GET /api/cost-management/v1/recommendations/openshift/cluster-quota/
GET /api/cost-management/v1/recommendations/openshift/cluster-quota/?format=csv&limit=100
GET /api/cost-management/v1/recommendations/openshift/cluster-quota/?filter[cluster]=550e8400-e29b-41d4-a716-446655440000
GET /api/cost-management/v1/recommendations/openshift/cluster-quota/?filter[cluster_quota_name]=team-payments-quota
GET /api/cost-management/v1/recommendations/openshift/cluster-quota/?filter[project]=team-payments-ns
GET /api/cost-management/v1/recommendations/openshift/cluster-quota/?filter[recommendation_type]=tighten&filter[risk_level]=high
GET /api/cost-management/v1/recommendations/openshift/cluster-quota/?group_by[cluster]=*&limit=20&offset=0
GET /api/cost-management/v1/recommendations/openshift/cluster-quota/?order_by=utilization&order_how=desc&limit=20&offset=0
GET /api/cost-management/v1/recommendations/openshift/cluster-
```

```
quota/?filter[tag:environment]=production
GET /api/cost-management/v1/recommendations/openshift/cluster-quota/detail?cluster_uuid=550e8400-e29b-41d4-a716-446655440001&cluster_quota_name=team-payments-quota
```

Available filters:

- `filter[cluster]` — cluster UUID (exact match)
- `filter[cluster_quota_name]` — CRQ object name (aliases: `filter[cluster_resource_quota]`, `filter[crq]`)
- `filter[project]` — namespace that belongs to the CRQ selector (`filter[namespace]` accepted as alias)
- `filter[recommendation_type]` — `tighten`, `raise`, `optimal`, or `none`
- `filter[risk_level]` — `high`, `medium`, `low`, or `none`
- `filter[tag:<key>]` — namespace label filter (requires `ROS_TAGS_ENABLED=true`; CRQs whose namespace selector includes a matching label)
- `group_by[cluster]` — aggregate rows per cluster (returns `count`, summed `estimated_savings`, and summed `capacity_freed` per cluster; mutually exclusive with per-CRQ list fields such as `cluster_quota_name`)
- `order_by` — `cluster_quota_name`, `utilization`, `risk_level`, or `estimated_monthly_savings`
- `order_how` — `asc` or `desc` (default `desc` when `order_by` is set)
- `limit` (1–100, default 20), `offset` (default 0)
- `after` — keyset cursor from `meta.next_cursor` (ignores `offset` when set); see pagination docs

Detail endpoint returns one CRQ recommendation object (not wrapped in `data`). The response includes a `history` array of per-resource snapshots (`recorded_at`, `resource`, `recommendation_type`, `risk_level`, and optional hard/used/utilization fields) for trend charts.

Example request:

```
curl -s -H "x-rh-identity: ${IDENTITY}" \
  'http://localhost:8000/api/cost-management/v1/recommendations/openshift/cluster-quota/?filter[recommendation_type]=tighten&limit=10'
```

Example response snippet:

```
{
  "meta": { "count": 1, "limit": 10, "offset": 0, "has_next": false, "currency": "USD" },
  "links": { "first": "...", "last": "..." },
  "data": [
    {
      "cluster_uuid": "550e8400-e29b-41d4-a716-446655440001",
      "cluster_quota_name": "team-payments-quota",
      "namespaces": ["namespace-a", "namespace-b"],
      "recommendation_type": "tighten",
      "risk_level": "low",
      "quota_hard": {
        "cpu_request_millicores": 500000,
        "memory_request_bytes": 1099511627776,
        "storage_request_bytes": 107374182400,
```

```
        "pods": 500
      },
      "quota_used": {
        "cpu_request_millicores": 175000,
        "storage_request_bytes": 53687091200,
        "pods": 120
      },
      "quota_recommended": {
        "cpu_request_millicores": 396000,
        "memory_request_bytes": 496125722624,
        "storage_request_bytes": 85899345920,
        "pods": 400
      },
      "utilization": {
        "cpu_request_percent": 35.0,
        "memory_request_percent": 12.0,
        "storage_request_percent": 50.0,
        "pods_percent": 24.0
      },
      "capacity_freed": {
        "cpu_cores_freed": 104,
        "memory_bytes": 603387187152,
        "storage_request_bytes": 21474836480,
        "pods_freed": 100
      },
      "estimated_savings": { "value": "420.00", "units": "USD" }
    }
  ]
}
```

Requires the `cluster-quota` plugin (`ROS_ENABLED_PLUGINS`); when disabled the route returns `404`. Clusters without ClusterResourceQuota objects return an empty `data` array (not an error).

Notification codes

CRQ rows may emit notification codes **70–73**. Filter the catalog with `GET /api/cost-management/v1/recommendations/openshift/notification-codes/?filter[plugin]=cluster-quota`.

Code	Name	Description
70	QUOTA_NEAR_CAPACITY	CRQ at high utilization risk
71	QUOTA_OVERSIZED	CRQ tighten recommendation generated
72	QUOTA_BLOCKING	CRQ resource blocking (used >= hard)
73	CLUSTER_QUOTA_AT_CAPACITY	CRQ at capacity alert

Object-count resources (`count/deployments.apps`, `count/secrets`, etc.) contribute to `risk_level` and blocking notifications (code 72) only — they are not exposed in API `utilization` fields or used in `order_by=utilization`.

When Koku cost model rates change, masu triggers `POST /api/cost-management/v1/internal/recalculate-savings` with `recommendation_types` including `cluster-quota` to refresh `estimated_savings` on existing tighten

rows.

Example `group_by[cluster]` response snippet:

```
{
  "meta": { "count": 2, "limit": 20, "offset": 0, "currency": "USD" },
  "data": [
    {
      "cluster_uuid": "550e8400-e29b-41d4-a716-446655440001",
      "count": 3,
      "capacity_freed": {
        "cpu_cores_freed": 12,
        "memory_bytes": 85899345920,
        "storage_request_bytes": 10737418240,
        "pods_freed": 5
      },
      "estimated_savings": { "value": "840.00", "units": "USD" }
    }
  ]
}
```

Cluster-quota recommendation settings (per-organization thresholds; env vars lock fields in `locked_fields`):

```
GET /api/cost-management/v1/recommendations/openshift/settings/cluster-quota
PUT /api/cost-management/v1/recommendations/openshift/settings/cluster-quota
DELETE /api/cost-management/v1/recommendations/openshift/settings/cluster-quota
```

Example GET/PUT body:

```
{
  "headroom_percent": 10,
  "high_risk_threshold_percent": 90,
  "medium_risk_threshold_percent": 70
}
```

PUT validation matches namespace quota settings. DELETE removes per-org overrides (falls back to `ROS_CLUSTER_QUOTA_*` env or defaults 10/90/70).

Snapshot staleness recommendations

VolumeSnapshot analysis — orphaned, stale, never-restored, redundant, and backup-managed snapshots with estimated storage holding cost. Requires the `snapshot` plugin in `ROS_ENABLED_PLUGINS`; when disabled, routes return `404`.

Snapshot list

```
GET /api/cost-management/v1/recommendations/openshift/snapshots
GET /api/cost-management/v1/recommendations/openshift/snapshots?limit=20&offset=0
GET /api/cost-management/v1/recommendations/openshift/snapshots?filter[cluster]=550e8400-e29b-
```

```
41d4-a716-446655440000
GET /api/cost-management/v1/recommendations/openshift/snapshots?filter[project]=production
GET /api/cost-
management/v1/recommendations/openshift/snapshots?filter[recommendation_type]=orphaned
GET /api/cost-
management/v1/recommendations/openshift/snapshots?filter[cluster]=CLUSTER_UUID&filter[project]=pa
yments-prod&filter[recommendation_type]=stale&limit=10&offset=0
```

Pagination: `limit` (1–100, default 20), `offset` (default 0). List rows are sorted by `age_days` descending (no `order_by` on this endpoint).

Filters (`filter[…]`):

Filter	Values / notes
`cluster`	Cluster UUID (RBAC-scoped)
`project`	Namespace (`filter[namespace]` accepted as alias)
`recommendation_type`	`orphaned`, `never_restored`, `redundant`, `stale`, `managed`, `active`

NOTE `filter[tag:key]` is not supported on snapshot endpoints.

Example list response snippet:


```
{
  "meta": { "count": 1, "limit": 20, "offset": 0, "currency": "USD" },
  "data": [
    {
      "cluster_uuid": "550e8400-e29b-41d4-a716-446655440000",
      "namespace": "production",
      "snapshot_name": "db-backup-2025-12-01",
      "source_pvc_name": "postgres-data",
      "volume_snapshot_class": "csi-aws-vsc",
      "storageclass": "gp3",
      "creation_timestamp": "2025-12-01T03:00:00Z",
      "restore_size_bytes": 53687091200,
      "age_days": 158,
      "source_pvc_exists": true,
      "restored_pvc_count": 0,
      "managed_by": "",
      "recommendation_type": "stale",
      "estimated_monthly_cost_usd": 2.5,
      "notifications": {
        "34": {
          "type": "INFO",
          "message": "Snapshot older than retention threshold with no known usage",
          "code": 34
        }
      }
    }
  ]
```

```
}
```

Snapshot summary

Namespace/cluster rollup for prioritizing cleanup (reclaimable cost and size exclude `active` and `managed`):

```
GET /api/cost-management/v1/recommendations/openshift/snapshots/summary
GET /api/cost-management/v1/recommendations/openshift/snapshots/summary?group_by=project
GET /api/cost-management/v1/recommendations/openshift/snapshots/summary?group_by=cluster
GET /api/cost-
management/v1/recommendations/openshift/snapshots/summary?filter[cluster]=CLUSTER_UUID
GET /api/cost-management/v1/recommendations/openshift/snapshots/summary?filter[project]=payments-
*
GET /api/cost-
management/v1/recommendations/openshift/snapshots/summary?filter[recommendation_type]=orphaned&gr
oup_by=project
GET /api/cost-
management/v1/recommendations/openshift/snapshots/summary?filter[recommendation_type]=stale&group
_by=namespace&limit=20&offset=0
GET /api/cost-
management/v1/recommendations/openshift/snapshots/summary?order_by=reclaimable_restore_size_gib&o
rder_how=desc&limit=20&offset=0
```

Summary filters (`filter[…]`):

Filter	Values / notes
`cluster`	Cluster UUID (RBAC-scoped)
`project`	Namespace (`filter[namespace]` accepted as alias); supports `*` wildcard
`recommendation_type`	`orphaned`, `never_restored`, `redundant`, `stale`, `managed`, `active` – restricts which snapshots are aggregated into each group (`counts_by_type` and reclaimable totals reflect only matching rows)

Summary `order_by`: `reclaimable_monthly_holding_cost_usd`, `reclaimable_restore_size_gib`, `actionable_snapshot_count`, `snapshot_count`. Use `order_how=asc` or `desc`.

Snapshot settings

Per-organization classification thresholds and cost rate (`locked_fields` lists env-locked keys):

```
GET /api/cost-management/v1/recommendations/openshift/settings/snapshot
PUT /api/cost-management/v1/recommendations/openshift/settings/snapshot
DELETE /api/cost-management/v1/recommendations/openshift/settings/snapshot
```

| Setting | Env var (locks value) | Default | Description | |---------|----------------------|---------|-------------| | `orphan_age_days` | `ROS_SNAPSHOT_ORPHAN_AGE_DAYS` | 7 | Orphan threshold; also active ceiling | | `never_restored_days` | `ROS_SNAPSHOT_NEVER_RESTORED_DAYS` | 30 | Never-restored threshold | | `stale_days` | `ROS_SNAPSHOT_STALE_DAYS` | 90 | Staleness and redundant age gate | | `redundant_threshold` |

`ROS_SNAPSHOT_REDUNDANT_THRESHOLD` | 3 | Max snapshots per PVC before redundant | | `cost_per_gib_month_usd` | `ROS_SNAPSHOT_COST_PER_GIB_MONTH` | 0.05 | Monthly $/GiB holding estimate | | `inventory_fresh_hours` | `ROS_SNAPSHOT_INVENTORY_FRESH_HOURS` | 6 | Hours of fresh inventory for classify/reconcile |

Example GET/PUT body:

```
{
  "orphan_age_days": 7,
  "never_restored_days": 30,
  "stale_days": 90,
  "redundant_threshold": 3,
  "cost_per_gib_month_usd": 0.05,
  "inventory_fresh_hours": 6,
  "locked_fields": []
}
```

DELETE returns **204** and removes per-org overrides (falls back to env or compiled defaults).

HCP correlation settings

Thin cross-plane correlator policy (requires the `hcp` plugin; when disabled the route returns `404`). PUT replaces the whole domain — all fields required — and values take effect on the next hourly correlator run (no recalculation; at most one cycle of delay):

```
GET /api/cost-management/v1/recommendations/openshift/settings/hcp-correlation
PUT /api/cost-management/v1/recommendations/openshift/settings/hcp-correlation
DELETE /api/cost-management/v1/recommendations/openshift/settings/hcp-correlation
```

| Setting | Env var (locks value) | Default | Description | |---------|----------------------|---------|-------------| | `h_p99_threshold_s` | `ROS_HCP_H_P99_THRESHOLD_S` | 0.3 | Absolute hosted API p99 floor (s); the H gate is `max(this, baseline_multiple x 7-day median)` | | `h_baseline_multiple` | `ROS_HCP_H_BASELINE_MULTIPLE` | 3 | Relative leg of the H gate against the 7-day median p99 | | `c_cpu_pct` | `ROS_HCP_C_CPU_PCT` | 80 | HCP-namespace trailing-day CPU usage/requests (%) firing C | | `c_etcd_p99_s` | `ROS_HCP_C_ETCD_P99_S` | 0.01 | etcd fsync p99 (s); stored and validated, strengthens C when wired | | `window_h` | `ROS_HCP_WINDOW_HOURS` | 1 | Hourly-aligned correlation window (h) | | `skew_m` | `ROS_HCP_SKEW_MINUTES` | 5 | Cross-plane clock-skew tolerance (minutes) | | `freshness_h` | `ROS_HCP_FRESHNESS_HOURS` | 2 | Evidence freshness (h); must cover the window (`freshness_h >= window_h`) | | `expiry_h` | `ROS_HCP_EXPIRY_HOURS` | 24 | Advisory expiry (h); expired advisories never render as current | | `z_idle_req_per_day` | `ROS_HCP_ZOMBIE_IDLE_REQ_PER_DAY` | 100 | DEPRECATED zombie idle threshold (accepted, unread — do not tune) | | `z_idle_cpu_floor_mc` | `ROS_HCP_ZOMBIE_IDLE_CPU_FLOOR_MC` | 10 | Daily per-workload CPU (mC) counting as active; discouraged to tune | | `z_short_window_days` | `ROS_HCP_ZOMBIE_SHORT_WINDOW_DAYS` | 3 | Pool-zero fast-lane window (d); validated ▮ idle window | | `z_short_required_days` | `ROS_HCP_ZOMBIE_SHORT_REQUIRED_DAYS` | 3 | Covered pool-evidence days; validated ▮ short window | | `z_idle_window_days` | `ROS_HCP_ZOMBIE_IDLE_WINDOW_DAYS` | 14 | Standard idle window (d); Christmas-docs apply — shorten with care |

Example GET/PUT body:

```
{
  "h_p99_threshold_s": 0.3,
  "h_baseline_multiple": 3,
  "c_cpu_pct": 80,
  "c_etcd_p99_s": 0.01,
  "window_h": 1,
  "skew_m": 5,
  "freshness_h": 2,
  "expiry_h": 24,
  "z_idle_req_per_day": 100,
  "z_idle_cpu_floor_mc": 10,
  "z_short_window_days": 3,
  "z_short_required_days": 3,
  "z_idle_window_days": 14,
  "locked_fields": []
}
```


PUT validation: `h_p99_threshold_s` 0.001–3600; `h_baseline_multiple` 1–100; `c_cpu_pct` 1–100; `c_etcd_p99_s` 0.001–60; `window_h` 1–168; `skew_m` 0–60; `freshness_h` 1–720 and `>= window_h`; `expiry_h` 1–720; `z_idle_req_per_day` 1–1e6; `z_idle_cpu_floor_mc` 1–100000; `z_short_window_days` 1–90 and `▮ z_idle_window_days`; `z_short_required_days` 1–90 and `▮ z_short_window_days`; `z_idle_window_days` 1–90. Unknown fields are rejected; env-locked fields return 403 with offending names. Fixed methodological floors (not tunable, no env var): 30-minute minimum data per window and 36-hour node-recency calibration for N. DELETE returns **204** and removes per-org overrides.

Filter the catalog with `GET /api/cost-management/v1/recommendations/openshift/notification-codes/?filter[plugin]=snapshot`.

Code	Name	Description
31	SNAPSHOT_ORPHANED	Source PVC was deleted; snapshot may no longer be needed
32	SNAPSHOT_NEVER_USED	Snapshot has never been used to restore a volume
33	SNAPSHOT_REDUNDANT	Newer snapshot exists for the same PVC
34	SNAPSHOT_STALE	Snapshot older than retention threshold with no known usage
35	SNAPSHOT_MANAGED	Snapshot is managed by backup tool — review retention policy

`active` classifications emit no snapshot notification code. List rows use a `notifications` map keyed by code string.

Snapshot age distribution

Histogram of snapshot counts grouped by age buckets. Requires Visual Insights enabled (`ROS_VISUAL_INSIGHTS_ENABLED=true`) and the snapshot plugin.

```
GET /api/cost-management/v1/recommendations/openshift/snapshots/age-distribution
GET /api/cost-management/v1/recommendations/openshift/snapshots/age-
distribution?bucket_boundaries=14,60
```

Optional: `bucket_boundaries` (comma-separated day thresholds, e.g. `14,60` creates `0-14`, `14-60`, `60+` buckets; defaults to `7,30,90` when omitted).

Response contains `buckets[]` (each with `label`, `min_days`, `max_days` — `null` for the last unbounded bucket — and `count`) and a `total` field.

Snapshot cost by type

Snapshot holding costs grouped by recommendation type. Requires Visual Insights enabled (`ROS_VISUAL_INSIGHTS_ENABLED=true`) and the snapshot plugin.

```
GET /api/cost-management/v1/recommendations/openshift/snapshots/cost-by-type
```

Response `data[]` contains `recommendation_type` (orphaned, stale, never_restored, redundant, managed, active), `total_cost_cents` (integer monthly holding cost), and `count`.

OpenShift Virtualization (VM) recommendations

VM-level CPU, memory, disk, and GPU sizing for KubeVirt workloads. Requires the ROS `vm` plugin (ensure `vm` is not in `ROS_DISABLED_PLUGINS` / Helm `enabledPlugins` includes `vm`). When disabled, VM routes return `404`.

Business hours are not applicable to VM recommendations (they apply to container and namespace recommendations only).

Monthly `estimated_monthly_savings` on list/detail requires `ROS_SAVINGS_ESTIMATES_ENABLED=true` (default) and `KOKU_MASU_URL` with cost model rates at ingestion. When disabled or masu is unreachable, `estimated_monthly_savings` is JSON `null`. Values are fixed at generation time until the next recommendation cycle (rate or currency changes are not retroactive). VM savings are **not** updated by `POST /internal/recalculate-savings` (see Savings recalculation (internal)); only a new VM ingest recalculates persisted dollar fields.

VM recommendations list

```
GET /api/cost-management/v1/recommendations/openshift/vm
GET /api/cost-management/v1/recommendations/openshift/vm?limit=20&offset=0
GET /api/cost-management/v1/recommendations/openshift/vm?filter[cluster]=CLUSTER_UUID
GET /api/cost-management/v1/recommendations/openshift/vm?filter[project]=my-namespace
GET /api/cost-management/v1/recommendations/openshift/vm?filter[vm_name]=my-vm
GET /api/cost-management/v1/recommendations/openshift/vm?filter[term]=medium_term
GET /api/cost-management/v1/recommendations/openshift/vm?filter[confidence]=high
GET /api/cost-management/v1/recommendations/openshift/vm?filter[is_idle]=true
GET /api/cost-management/v1/recommendations/openshift/vm?filter[is_abandoned]=true
GET /api/cost-management/v1/recommendations/openshift/vm?filter[is_oversized]=true
GET /api/cost-management/v1/recommendations/openshift/vm?filter[guest_agent_detected]=true
```

```
GET /api/cost-management/v1/recommendations/openshift/vm?filter[has_gpu]=true
GET /api/cost-management/v1/recommendations/openshift/vm?filter[gpu_classification]=underutilized
GET /api/cost-management/v1/recommendations/openshift/vm?filter[engine]=cost
GET /api/cost-management/v1/recommendations/openshift/vm?filter[engine]=performance
GET /api/cost-management/v1/recommendations/openshift/vm?filter[guest_os]=windows
GET /api/cost-management/v1/recommendations/openshift/vm?filter[is_network_bound]=true
GET /api/cost-management/v1/recommendations/openshift/vm?filter[tag:environment]=production
GET /api/cost-
management/v1/recommendations/openshift/vm?filter[tag:environment]=production,staging&filter[tag:
team]=platform
GET /api/cost-management/v1/recommendations/openshift/vm?limit=20&after=<meta.next_cursor>
```

`filter[tag:<key>]` requires `ROS_TAGS_ENABLED=true` (namespace labels synced from Koku). Multiple `filter[tag:*]` keys are ANDed.

Each list row includes `metadata.engine` matching the filter (one row per VM per engine when unfiltered).

Pagination: `limit` (1–100, default 10), `offset` (default 0), or keyset via `after` from `meta.next_cursor` (ignores `offset` when set).

Filters (`filter[…]` or legacy query names where noted):

Filter	Values / notes
`cluster`	Cluster UUID (RBAC-scoped)
`namespace`	VM namespace
`vm_name`	KubeVirt VM name
`term`	`short_term`, `medium_term`, or `long_term`
`engine`	`cost` or `performance`
`confidence`	`high`, `moderate`, or `low`
`is_idle`, `is_abandoned`, `is_oversized`, `guest_agent_detected`, `has_gpu`, `is_network_bound`	`true` or `false`
`gpu_classification`	Comma-separated GPU classes (e.g. `idle`, `underutilized`)
`guest_os`	Guest OS substring match, case-insensitive; comma-separated values are ORed (e.g. `linux`, `windows`, `linux,windows`)
`tag:<key>`	Namespace label filter when `ROS_TAGS_ENABLED=true` (`filter[tag:environment]=production`)

Each list item includes `metadata.is_network_bound` (`true` when sustained network metrics qualify the VM for **n1** network-optimized instance types). Use `filter[is_network_bound]=true` or `false` to narrow the list server-side.

Power-off metadata: `metadata.is_power_off_candidate` (code **64**) — VM is idle on many observed days but not

permanently abandoned; `metadata.power_off_idle_pct` (integer, nullable) — percent of observed days classified as idle.

Placement metadata (no list filters yet): `metadata.is_redundant_placement` (code **60**), `metadata.has_shared_storage` (code **62**), `metadata.numa_oversized` (code **63**). Code **61** is uneven node distribution (notification only).

`order_by` (default `vm_name`): `vm_name`, `namespace`, `current_vcpu`, `current_memory_gib`, `guest_os`, `recommended_vcpu`, `recommended_memory_gib`, `is_idle`, `is_abandoned`, `is_oversized`, `confidence`, `estimated_monthly_savings`, `last_recommended_at`. Use `order_how=asc` or `desc`. (`savings` and `savings_amount` accepted as deprecated aliases.)

VM detail

```
GET /api/cost-
management/v1/recommendations/openshift/vm/detail?cluster_uuid=UUID&vm_name=NAME&namespace=NS
GET /api/cost-
management/v1/recommendations/openshift/vm/detail?cluster_uuid=UUID&vm_name=NAME&namespace=NS&ter
m=medium_term&engine=cost
```

Required: `cluster_uuid`, `vm_name`, `namespace`. Optional: `term` (default `medium_term`), `engine` (default `cost`).

`io_profile.pattern` classifies disk I/O from peak p95 throughput and IOPS: `sequential`, `random`, `mixed`, `low-io`, or empty when metrics are missing. Codes **58** and **59** align with `sequential` and `random` patterns.

Example detail response (CPU/memory/disk sizing in `current` / `recommended`; GPU in `gpu`; time series in `daily_digests`; optional sibling `daily_digests_business_hours` when Visual Insights is on):


```
{
  "vm_name": "my-vm",
  "namespace": "my-namespace",
  "cluster_uuid": "550e8400-e29b-41d4-a716-446655440000",
  "guest_os": "linux",
  "current": {
    "vcpu": 4,
    "memory_gib": 16,
    "disk_gib": 100,
    "instance_type": "u1.medium"
  },
  "recommended": {
    "vcpu": 2,
    "memory_gib": 8,
    "disk_gib": 100,
    "instance_type": "n1.medium",
    "series": "n1"
  },
  "metadata": {
    "guest_agent_detected": true,
    "confidence": "high",
    "term": "medium_term",
    "engine": "cost",
    "is_idle": false,
```

```
    "is_abandoned": false,
    "is_power_off_candidate": false,
    "is_oversized": true,
    "is_network_bound": true,
    "is_redundant_placement": false,
    "has_shared_storage": false,
    "numa_oversized": false,
    "preference_name": "small",
    "preference_class": "virtualmachineclusterpreference"
  },
  "io_profile": {
    "read_iops_p95": 1200,
    "write_iops_p95": 800,
    "read_bps_p95": 125829120,
    "write_bps_p95": 83886080,
    "hint": "balanced",
    "pattern": "sequential"
  },
  "disk_projection": {
    "days_until_full": 45,
    "growth_gib_per_day": 0.12,
    "recommended_expand_gib": 0
  },
  "notifications": [
    {
      "code": 55,
      "type": "warning",
      "message": "Network-saturated workload: recommend n1 network-optimized instance type"
    }
  ],
  "savings": {
    "value": "18104.00",
    "units": "USD"
  },
  "gpu": {
    "gpu_count": 1,
    "gpu_model": "NVIDIA-A100",
    "gpu_classification": "underutilized",
    "recommended_gpu_action": "time_slice",
    "recommended_time_slice_count": 4,
    "gpu_timeslice_confidence": "high",
    "gpu_timeslice_rationale": "7 days of observations; peak SM/DRAM/FB fractions support 4
replicas",
    "recommended_vgpu_profile": "nvidia.com/gpu.a100-4c.8g",
    "gpu_devices": []
  },
  "last_recommended_at": "2026-05-01T12:00:00Z",
  "daily_digests": [
    {
      "bucket_date": "2026-04-30",
      "cpu_usage_p95_mc": 1800,
      "mem_usage_p95_kib": 8388608,
      "sample_count": 24
    }
  ]
```

```
}
```

VM recommendation history

Per-VM history of past sizing recommendations:

```
GET /api/cost-
management/v1/recommendations/openshift/vms/{vm_name}/history?cluster_uuid=UUID&namespace=NS&limi
t=20&offset=0
GET /api/cost-
management/v1/recommendations/openshift/vms/{vm_name}/history?cluster_uuid=UUID&namespace=NS&term
=short_term&engine=cost
```

Required query params: `cluster_uuid` (or `cluster_id`), `namespace`. Path param: `vm_name`. Optional: `term` (default `short_term`), `engine` (default `cost`), `limit`, `offset`.

Retention: `ROS_VM_REC_HISTORY_RETENTION_DAYS` (default **90** days) controls how long `vm_recommendation_history` snapshots are kept.

VM hourly activity

Hourly CPU, memory, and disk I/O digests for a single VM, enabling heatmap and activity pattern charts. Requires Visual Insights enabled (`ROS_VISUAL_INSIGHTS_ENABLED=true`) and hourly VM digests enabled (`ROS_HOURLY_VM_DIGESTS_ENABLED=true`).

```
GET /api/cost-management/v1/recommendations/openshift/vm/hourly-
activity?cluster_uuid=CLUSTER_UUID&vm_name=my-vm&namespace=my-namespace
GET /api/cost-management/v1/recommendations/openshift/vm/hourly-
activity?cluster_uuid=CLUSTER_UUID&vm_name=my-vm&namespace=my-namespace&days=30
```

Required: `cluster_uuid` (or `filter[cluster]`), `vm_name`, `namespace`. Optional: `days` (default 14, max 90).

Each data point returns `report_date`, `hour` (0–23), `cpu_usage_p95_mc`, `mem_usage_p95_kib`, `sample_count`, `disk_read_iops_p95`, `disk_write_iops_p95`. Data is stored in `hourly_vm_digests`; retention controlled by `ROS_HOURLY_VM_DIGESTS_RETENTION_DAYS` (default 90).

CSV export

Flattened CSV export for VM list and per-VM history (same filters as JSON):

```
GET /api/cost-management/v1/recommendations/openshift/vm?format=csv
GET /api/cost-
management/v1/recommendations/openshift/vm?format=csv&limit=100&order_by=estimated_monthly_saving
s&order_how=desc
GET /api/cost-
management/v1/recommendations/openshift/vms/{vm_name}/history?cluster_uuid=UUID&namespace=NS&form
at=csv
```

Prefer `Accept: text/csv` for explicit content negotiation. Response `Content-Type` is `text/csv`.

VM instance types

Catalog of instance types and VM preference summary for a cluster:

```
GET /api/cost-management/v1/recommendations/openshift/instance-types?cluster_uuid=UUID
```

VM settings

Dedicated VM threshold, disk, I/O, memory floor, and stability settings (not the generic `/settings/thresholds` endpoint):

```
GET /api/cost-management/v1/recommendations/openshift/settings/vm
PUT /api/cost-management/v1/recommendations/openshift/settings/vm
DELETE /api/cost-management/v1/recommendations/openshift/settings/vm
```

Example GET response:

```
{
  "enabled": true,
  "cpu_adaptive_margin_enabled": true,
  "history_retention_days": 90,
  "thresholds": {
    "cpu_percentile_cost": 0.60,
    "cpu_percentile_perf": 0.98,
    "cpu_margin_min": 1.15,
    "cpu_margin_max": 1.50,
    "mem_margin_min": 1.10,
    "downsize_hysteresis_ratio": 0.85,
    "min_vcpu_change": 1,
    "min_gib_change": 1,
    "idle_cpu_mc": 50,
    "idle_memory_mib": 256,
    "idle_cpu_mc_windows": 100,
    "idle_memory_mib_windows": 512,
    "abandoned_min_days": 14
  },
  "memory_floors": {
    "linux_gib": 2,
    "windows_gib": 4,
    "windows_kernel_reserve_gib": 0.5
  },
  "stability": {
    "downsize_stability_days": 3,
    "crash_loop_restart_threshold": 5
  },
  "disk": {
    "projection_window_days": 14,
    "headroom_pct": 0.20,
    "round_step_gib": 10,
```

```
    "min_growth_mib_per_day": 10
  },
  "io": {
    "high_iops_threshold": 3000,
    "sequential_threshold_bytes": 65536,
    "random_threshold_bytes": 16384,
    "min_iops_for_classification": 100
  },
  "network": {
    "throughput_threshold_bps": 62500000,
    "pps_threshold": 100000,
    "drop_ratio_bp": 10,
    "sustained_days": 7,
    "enable_network_series": true
  },
  "gpu": {
    "idle_threshold_bp": 500,
    "underutil_threshold_bp": 3000,
    "fb_saturation_mib": 0,
    "compute_saturation_threshold_bp": 8500,
    "gpu_timeslice_min_replicas": 2,
    "gpu_timeslice_max_replicas": 16,
    "gpu_timeslice_fb_safety_threshold_bp": 8000,
    "gpu_timeslice_dram_penalty_threshold_bp": 5000
  },
  "placement": {
    "enable_placement_checks": true,
    "placement_skew_ratio": 3,
    "enable_shared_pvc_correlation": true,
    "numa_node_memory_gib": 64
  },
  "instance_type_matching": true,
  "locked_fields": [],
  "settings_locked": false
}
```

Optional settings sub-blocks (partial PUT supported):

- `power_schedule` – simplified power-off detection (notification **64**); env `ROS_VM_ENABLE_POWER_SCHEDULE`, `ROS_VM_POWER_OFF_*`
- `network_qos` – SR-IOV / DPDK hints for network-bound VMs (notifications **65**–**66**); env `ROS_VM_NETWORK_QOS_*`
- `storage_tiering` – multi-day I/O tier hints (notifications **67**–**69**); env `ROS_VM_STORAGE_TIERING_*`

`placement` – cluster placement, correlated workloads, and NUMA fit (notifications **60**–**63**):

- `enable_placement_checks` – same-node redundancy and node skew (default `true`)
- `placement_skew_ratio` – max:min VM count per node before skew notification **61** (default `3`)
- `enable_shared_pvc_correlation` – correlated profile peers in namespace (default `true`)
- `numa_node_memory_gib` – single-NUMA memory cap for code **63** (default `64`)

Env locks: `ROS_VM_ENABLE_PLACEMENT_CHECKS`, `ROS_VM_PLACEMENT_SKEW_RATIO`, `ROS_VM_ENABLE_SHARED_PVC_CORRELATION`, `ROS_VM_NUMA_NODE_MEMORY_GIB`.

`network` — n1 network-optimized classification (defaults shown above):

- `throughput_threshold_bps` — sustained aggregate rx+tx P95 above this (bytes/sec; default `62500000` ▌500 Mbps)
- `pps_threshold` — packets-per-second P95 threshold for alternate network-bound path (default `100000`)
- `drop_ratio_bp` — max daily drop ratio must exceed this in basis points (default `10` = 0.1%) with high PPS
- `sustained_days` — days in the term window meeting throughput or PPS+drop criteria (default `7`)
- `enable_network_series` — when `true`, network-bound VMs may receive **n1.*** instance types (default `true`)

`io` — disk I/O profiling and pattern classification:

- `high_iops_threshold` — total read+write p95 IOPS above this triggers code **39** (`VM_HIGH_IO`) (default `3000`)
- `sequential_threshold_bytes` — average I/O size (peak BPS / peak IOPS) ▌this ▌`io_profile.pattern` `sequential` and code **58** (default `65536` = 64 KiB)
- `random_threshold_bytes` — average I/O size < this ▌`random` and code **59** (default `16384` = 16 KiB); values between random and sequential thresholds yield `mixed`
- `min_iops_for_classification` — peak IOPS below this ▌`low-io` (no pattern codes) (default `100`)

`gpu` — VM GPU classification and vGPU time-slicing tunables:

- `idle_threshold_bp` — SM utilization below this ▌idle classification (default `500` = 5%)
- `underutil_threshold_bp` — below this but above idle ▌underutilized (default `3000` = 30%)
- `fb_saturation_mib` — frame-buffer saturation threshold in MiB; `0` auto-detects from GPU catalog (default `0`)
- `compute_saturation_threshold_bp` — SM fraction at or above this ▌compute-saturated (default `8500` = 85%)
- `gpu_timeslice_min_replicas` — minimum `recommended_time_slice_count` when time-slicing is advised (default `2`)
- `gpu_timeslice_max_replicas` — upper cap on slice count; reduced when DRAM exceeds penalty threshold (default `16`)
- `gpu_timeslice_fb_safety_threshold_bp` — do not recommend time-slicing when frame-buffer fraction ▌this (default `8000` = 80%)
- `gpu_timeslice_dram_penalty_threshold_bp` — when DRAM ▌this, max replicas is halved (default `5000` = 50%)

Detail responses may include GPU fields `gpu_timeslice_confidence` (`high`, `moderate`, `low`, or empty), `gpu_timeslice_rationale`, and `recommended_vgpu_profile` when time-slicing or vGPU profile selection applies.

Example PUT (partial update — I/O pattern, GPU classification, network, and time-slicing):

```
{
  "thresholds": {
    "cpu_margin_min": 1.18
  },
```

```
  "io": {
    "sequential_threshold_bytes": 131072,
    "random_threshold_bytes": 8192,
    "min_iops_for_classification": 200
  },
  "network": {
    "throughput_threshold_bps": 100000000,
    "sustained_days": 5
  },
  "gpu": {
    "idle_threshold_bp": 300,
    "underutil_threshold_bp": 2500,
    "compute_saturation_threshold_bp": 9000,
    "gpu_timeslice_max_replicas": 8,
    "gpu_timeslice_fb_safety_threshold_bp": 7500
  }
}
```

`history_retention_days` is read-only (set by `ROS_VM_REC_HISTORY_RETENTION_DAYS`). DELETE removes per-org VM settings overrides (`204 No Content`).

VM notification codes

VM list/detail `notifications` arrays use lowercase `type` (`info`, `warning`, `critical`) and numeric `code`. The full catalog spans **18–69** for the `vm` plugin:

Range	Topics
18–19	Idle (**18**), oversized (**19**)
37–54	Sizing, disk, guest agent, GPU, crash-loop, instance types
55–59	Network-bound (**55**), vGPU/time-slice (**56**–**57**), I/O pattern (**58**–**59**)
60–63	Placement and NUMA (**60**–**63**)
64	Power-off scheduling candidate
65–66	Network QoS (SR-IOV / DPDK hints)
67–69	Storage tiering hints

`GET /api/cost-management/v1/recommendations/openshift/notification-codes?filter[plugin]=vm` returns all VM codes with names and messages. See also the public notification catalog in ros-ocp-backend docs.

Examples (network, I/O, placement):

Code	Name	Severity	When emitted
55	`VM_NETWORK_SATURATED`	warning	`metadata.is_network_bound` is `true` – recommend **n1** network-optimized instance type

Code	Name	Severity	When emitted
56	`VM_VGPU_PROFILE_RECOMMENDED`	info	vGPU profile recommended — see `recommended_vgpu_profile` in the `gpu` object
57	`VM_GPU_TIMESLICE_UNSAFE_FB`	warning	Frame-buffer usage too high for safe shared vGPU time-slicing
58	`VM_IO_SEQUENTIAL`	info	`io_profile.pattern` is `sequential` — consider storage optimized for throughput
59	`VM_IO_RANDOM`	info	`io_profile.pattern` is `random` — consider storage optimized for IOPS
60	`VM_REDUNDANT_COLOCATION`	warning	Redundant VMs co-located on same node — consider adding anti-affinity rules (`metadata.is_redundant_placement` is `true`)
61	`VM_UNEVEN_NODE_DISTRIBUTION`	info	Uneven VM distribution across nodes — consider topologySpreadConstraints
62	`VM_SHARED_STORAGE`	info	VM shares storage with other VMs — correlated workload group detected (`metadata.has_shared_storage` is `true`)
63	`VM_NUMA_OVERSIZED`	warning	VM memory exceeds single NUMA node capacity — NUMA pinning not possible (`metadata.numa_oversized` is `true`)

VM term settings

Separate from generic `/settings/terms` — controls short/medium/long windows for VM recommendations only:

```
GET /api/cost-management/v1/recommendations/openshift/settings/vm/terms
PUT /api/cost-management/v1/recommendations/openshift/settings/vm/terms
DELETE /api/cost-management/v1/recommendations/openshift/settings/vm/terms
```

Example GET response:

```
{
  "terms": [
    {
      "name": "short_term",
      "window_days": 1,
      "min_data_days": 1,
      "locked": false,
      "is_default": true
    },
    {
      "name": "medium_term",
```

```
      "window_days": 7,
      "min_data_days": 3,
      "locked": false,
      "is_default": true
    },
    {
      "name": "long_term",
      "window_days": 15,
      "min_data_days": 7,
      "locked": false,
      "is_default": true
    }
  ],
  "settings_locked": false
}
```

Recommendation term settings

Customers can configure their own 3 recommendation term windows (1–90 days each), replacing the fixed short (1d) / medium (7d) / long (15d) defaults:

```
GET /api/cost-
management/v1/recommendations/openshift/settings/terms?recommendation_type={container|namespace|n
ode|gpu|pvc}
PUT /api/cost-
management/v1/recommendations/openshift/settings/terms?recommendation_type={container|namespace|n
ode|gpu|pvc}
DELETE /api/cost-
management/v1/recommendations/openshift/settings/terms?recommendation_type={container|namespace|n
ode|gpu|pvc}
```

Required query parameter `recommendation_type` selects the plugin whose short/medium/long windows are read or updated. VM term windows use `/settings/vm/terms` instead. Use `GET /settings/capabilities` to see which types are enabled and support terms.

Settings capabilities

Discover which recommendation plugins are enabled for the deployment and whether each supports configurable terms. Use each item's `name` as `recommendation_type` for `/settings/terms`. The top-level `business_hours` flag indicates whether business-hours settings endpoints are available (`ROS_BUSINESS_HOURS_ENABLED`).

```
GET /api/cost-management/v1/recommendations/openshift/settings/capabilities
```

GET response:

```
{
  "recommendation_types": [
    {
      "name": "container",
```

```
      "enabled": true,
      "supports_terms": true
    },
    {
      "name": "namespace",
      "enabled": true,
      "supports_terms": true
    },
    {
      "name": "pvc",
      "enabled": true,
      "supports_terms": true
    }
  ],
  "business_hours": false
}
```

Platform settings lock

When `ROS_SETTINGS_LOCKED=true`, tenant settings overrides are ignored on read and all settings `PUT`/`DELETE` requests return `403 Forbidden`:

```
{
  "error": "settings are locked by platform administrator",
  "locked": true
}
```

GET responses include `settings_locked: true` and `locked_fields: ["*"]`. Per-plugin opt-outs use `ROS_SETTINGS_LOCKED_VM`, `ROS_SETTINGS_LOCKED_TERMS`, etc. VM sizing thresholds use the dedicated `/settings/vm` endpoint (not `/settings/thresholds?recommendation_type=vm`).

Recommendation threshold settings

Per-organization sizing and classification thresholds for container, namespace, node, GPU, and PVC plugins. OpenShift Virtualization (VM) thresholds are configured via `/settings/vm` (see OpenShift Virtualization (VM) recommendations), not these endpoints.

Each plugin has a dedicated path (no query parameter). PUT accepts partial updates; DELETE removes tenant overrides so environment and compiled defaults apply. Fields locked by environment variables are listed in `locked_fields` and return `403` if modified via the API.

```
GET /api/cost-management/v1/recommendations/openshift/settings/container
PUT /api/cost-management/v1/recommendations/openshift/settings/container
DELETE /api/cost-management/v1/recommendations/openshift/settings/container

GET /api/cost-management/v1/recommendations/openshift/settings/namespace
PUT /api/cost-management/v1/recommendations/openshift/settings/namespace
DELETE /api/cost-management/v1/recommendations/openshift/settings/namespace

GET /api/cost-management/v1/recommendations/openshift/settings/node
```

```
PUT /api/cost-management/v1/recommendations/openshift/settings/node
DELETE /api/cost-management/v1/recommendations/openshift/settings/node

GET /api/cost-management/v1/recommendations/openshift/settings/gpu
PUT /api/cost-management/v1/recommendations/openshift/settings/gpu
DELETE /api/cost-management/v1/recommendations/openshift/settings/gpu

GET /api/cost-management/v1/recommendations/openshift/settings/pvc
PUT /api/cost-management/v1/recommendations/openshift/settings/pvc
DELETE /api/cost-management/v1/recommendations/openshift/settings/pvc
```

Deprecated alias (still supported; responses include `Deprecation: true` and a `Link` header pointing to the canonical path above):

```
GET /api/cost-
management/v1/recommendations/openshift/settings/thresholds?recommendation_type={container|namesp
ace|node|gpu|pvc}
PUT /api/cost-
management/v1/recommendations/openshift/settings/thresholds?recommendation_type={container|namesp
ace|node|gpu|pvc}
DELETE /api/cost-
management/v1/recommendations/openshift/settings/thresholds?recommendation_type={container|namesp
ace|node|gpu|pvc}
```

Container and namespace (`/settings/container` or `/settings/namespace`)

Container and namespace share the same response shape. Namespace defaults use a higher `mem_trend_slope_threshold` (500.0 vs 100.0 for containers).

GET/PUT response (`/settings/container`):

```
{
  "cpu_cost_percentile": 0.60,
  "cpu_perf_percentile": 0.98,
  "mem_cost_percentile": 0.95,
  "mem_perf_percentile": 1.0,
  "min_margin": 1.15,
  "max_margin": 1.50,
  "limit_multiplier": 1.05,
  "cpu_floor_mc": 25,
  "idle_cpu_threshold_mc": 10,
  "idle_mem_threshold_kib": 10240,
  "mem_trend_slope_threshold": 100.0,
  "low_confidence_threshold": 0.5,
  "sparse_data_threshold": 2,
  "locked_fields": []
}
```

GET response (`/settings/namespace`) – same fields, higher memory trend threshold:

```
{
```

```
  "cpu_cost_percentile": 0.60,
  "cpu_perf_percentile": 0.98,
  "mem_cost_percentile": 0.95,
  "mem_perf_percentile": 1.0,
  "min_margin": 1.15,
  "max_margin": 1.50,
  "limit_multiplier": 1.05,
  "cpu_floor_mc": 25,
  "idle_cpu_threshold_mc": 10,
  "idle_mem_threshold_kib": 10240,
  "mem_trend_slope_threshold": 500.0,
  "low_confidence_threshold": 0.5,
  "sparse_data_threshold": 2,
  "locked_fields": []
}
```

Example PUT (container — override CPU percentile):

```
{
  "cpu_cost_percentile": 0.65
}
```

Node (`/settings/node`)

```
{
  "underutil_threshold": 0.30,
  "overcommit_threshold": 1.50,
  "allocatable_factor": 0.93,
  "stranded_imbalance_threshold": 0.60,
  "ema_alpha": 0.30,
  "cost_target_utilization": 0.80,
  "perf_target_utilization": 0.55,
  "perf_consolidation_headroom_multiplier": 2.0,
  "trend_min_days": 3,
  "zombie_cpu_p95_mc": 200,
  "zombie_max_pods": 5,
  "idle_cpu_util_pct": 10,
  "idle_mem_util_pct": 10,
  "idle_max_pods": 10,
  "pod_headroom_consolidation_gate": 0.15,
  "pod_headroom_notification_threshold": 0.10,
  "locked_fields": []
}
```

Example PUT (node — override underutilization threshold):

```
{
  "underutil_threshold": 0.35
}
```

GPU (`/settings/gpu`)

```
{
  "idle_threshold": 0.02,
  "underutilized_sm_threshold": 0.25,
  "underutilized_tensor_threshold": 0.15,
  "membound_dram_threshold": 0.60,
  "membound_tensor_threshold": 0.15,
  "fb_headroom_factor": 1.20,
  "compute_bound_dram_threshold": 0.30,
  "mig_fb_percentile": 0.98,
  "confidence_days_tier1": 3,
  "confidence_days_tier2": 7,
  "confidence_days_tier3": 14,
  "spike_ratio_threshold": 5.0,
  "spike_confidence_penalty": 0.70,
  "no_profiling_confidence_factor": 0.50,
  "timeslicing_majority_threshold": 0.50,
  "timeslicing_min_replicas": 2,
  "timeslicing_max_replicas": 8,
  "timeslicing_base_penalty": 0.70,
  "timeslicing_impacted_weight": 0.30,
  "node_freshness_days": 7,
  "locked_fields": []
}
```

Example PUT (GPU — override idle threshold):

```
{
  "idle_threshold": 0.03
}
```

PVC (`/settings/pvc`)

```
{
  "oversized_threshold": 0.20,
  "near_full_threshold": 0.85,
  "min_trend_days": 2,
  "recommended_size_multiplier": 2,
  "min_recommended_gib": 1,
  "days_to_full_alert": 30,
  "locked_fields": []
}
```

Example PUT (PVC — override oversized threshold):

```
{
  "oversized_threshold": 0.25
}
```

Business hours settings

Optional schedule-aware sizing (CPU requests/limits and memory requests/limits) (requires `ROS_BUSINESS_HOURS_ENABLED` and `business_hours: true` in `/recommendations/openshift/settings/capabilities`). Enabling a schedule triggers digest re-shipping for affected scopes and approximately doubles digest storage.

Org-default schedule:

```
GET /api/cost-management/v1/recommendations/openshift/settings/business-hours
PUT /api/cost-management/v1/recommendations/openshift/settings/business-hours
DELETE /api/cost-management/v1/recommendations/openshift/settings/business-hours
```

Cluster override (`:cluster_id` is the cluster UUID):

```
GET /api/cost-management/v1/recommendations/openshift/settings/business-
hours/clusters/:cluster_id
PUT /api/cost-management/v1/recommendations/openshift/settings/business-
hours/clusters/:cluster_id
DELETE /api/cost-management/v1/recommendations/openshift/settings/business-
hours/clusters/:cluster_id
```

Namespace override (most specific wins):

```
GET /api/cost-management/v1/recommendations/openshift/settings/business-
hours/clusters/:cluster_id/namespaces/:namespace
PUT /api/cost-management/v1/recommendations/openshift/settings/business-
hours/clusters/:cluster_id/namespaces/:namespace
DELETE /api/cost-management/v1/recommendations/openshift/settings/business-
hours/clusters/:cluster_id/namespaces/:namespace
```

Effective schedule (resolved inheritance; optional `cluster_id` and `namespace` query parameters):

```
GET /api/cost-management/v1/recommendations/openshift/settings/business-
hours/effective?cluster_id=:cluster_id&namespace=:namespace
```

GET response (example — namespace override wins):

```
{
  "timezone": "America/New_York",
  "schedule": {
    "days": ["monday", "tuesday", "wednesday", "thursday", "friday"],
    "start_time": "10:00",
    "end_time": "15:00"
  },
  "off_hours_weight": 0.0,
  "enabled": true,
  "resolved_from": "namespace"
```

```
}
```

`resolved_from` is `namespace`, `cluster`, `org`, or `none` (no schedule applies). `timezone` uses an **IANA** name (for example `America/New_York`); `start_time` and `end_time` are interpreted in that timezone's local wall clock, not as UTC.

When `ROS_SETTINGS_LOCKED_BUSINESS_HOURS=true` (default when the global settings lock is on), business-hours PUT and DELETE return `403`. GET still returns the schedule with `settings_locked: true` and `enabled: false`.

PUT returns `202 Accepted` and triggers asynchronous digest reship (`reship_ros`). Cluster GET responses may include `reship_status`: `complete`, `pending`, or `forward_only` (degraded — forward data only). Org-level GET does not include `reship_status`.

Overnight windows are allowed: `schedule.end_time` may be before `schedule.start_time` (for example Mon–Fri `22:00`–`06:00`). The window is half-open `[start, end)` on the **local wall clock** in the IANA timezone (not elapsed duration; DST may skip or repeat an hour). Samples after midnight belong to the previous calendar day's shift (Friday's shift includes Saturday morning until `end_time`). Equal start and end times return `400`. `08:00`–`17:00` in the example below is a typical office window, not the only legal window. PUT may include a non-fatal wrap warning.

`enabled: false` keeps the override row but disables business-hours digest generation for that scope. DELETE removes the override so the scope inherits from cluster or org default.

Example PUT body:

```
{
  "timezone": "America/New_York",
  "schedule": {
    "days": ["monday", "tuesday", "wednesday", "thursday", "friday"],
    "start_time": "08:00",
    "end_time": "17:00"
  },
  "off_hours_weight": 0.0,
  "enabled": true
}
```

`days` uses lowercase English day names. `start_time` / `end_time` are 24-hour `HH:MM` in the configured IANA timezone. Container and namespace recommendation detail payloads include a nested `recommendation_engines.*.business_hours` sizing block when a schedule applies and reship is complete (savings always use the `all_hours` perspective).

Idle and zombie detection

Classifies containers as `active`, `idle` (low request-relative utilization), or `zombie` (near-zero CPU with sustained observation). Idle workloads expose waste and terminate guidance on list/detail responses; rightsizing savings are omitted when `idle_state` is not `active`.

Idle detection settings

Per-organization idle detection configuration. PUT accepts partial updates inside `idle_detection`; DELETE removes tenant overrides and restores environment/compiled defaults. Fields locked by environment variables appear in `locked_fields` and return `403` if modified via the API.

```
GET /api/cost-management/v1/recommendations/openshift/settings/idle-detection
PUT /api/cost-management/v1/recommendations/openshift/settings/idle-detection
DELETE /api/cost-management/v1/recommendations/openshift/settings/idle-detection
```

GET response:

```
{
  "idle_detection": {
    "enabled": true,
    "thresholds": {
      "cpu_utilization_percent": 2,
      "memory_utilization_percent": 5,
      "burst_ratio": 10,
      "minimum_observation_days": 14,
      "gpu_sm_active_basis_points": 500,
      "gpu_dram_active_basis_points": 500
    },
    "exclusions": {
      "namespaces": ["kube-system", "openshift-*"],
      "workload_types": ["DaemonSet"]
    }
  },
  "locked_fields": []
}
```

Example PUT (enable idle detection with custom thresholds and exclusions):

```
{
  "idle_detection": {
    "enabled": true,
    "thresholds": {
      "cpu_utilization_percent": 2,
      "memory_utilization_percent": 5,
      "burst_ratio": 10,
      "minimum_observation_days": 14,
      "gpu_sm_active_basis_points": 500,
      "gpu_dram_active_basis_points": 500,
      "zombie_cpu_millicores": 1,
      "zombie_peak_millicores": 10
    },
    "exclusions": {
      "namespaces": ["kube-system", "openshift-*"],
      "workload_types": ["DaemonSet"]
    }
  }
```

```
}
```

Container list response fields

When idle detection is enabled, each container recommendation row may include:

Field	Definition
idle_state	Classification: `active`, `idle`, or `zombie`
idle_since	ISO date (`YYYY-MM-DD`) when the workload entered the current idle state
idle_duration_days	Days in the current `idle_state`
peak_cpu_millicores	Peak CPU usage (millicores) observed in the observation window
peak_memory_bytes	Peak memory usage (bytes) observed in the observation window
estimated_monthly_waste	Monthly cost of retained resources for idle/zombie workloads (same `{value, units}` shape as savings fields; 6-decimal string + `USD`)
idle_recommendation	Terminate guidance when `idle_state` is `idle` or `zombie` (`action`, `confidence`, `reason`)

Example list row (idle/zombie):

```
{
  "idle_state": "zombie",
  "idle_since": "2026-04-15",
  "idle_duration_days": 42,
  "peak_cpu_millicores": 2,
  "peak_memory_bytes": 52428800,
  "estimated_monthly_waste": {
    "value": "89.50",
    "units": "USD"
  },
  "idle_recommendation": {
    "action": "terminate",
    "confidence": "high",
    "reason": "No meaningful CPU or memory activity for 42 days"
  }
}
```

Omit `idle_recommendation` when `idle_state=active`. Omit `estimated_monthly_waste` when savings estimates are disabled or cost data is unavailable. Omit `estimated_monthly_savings` when `idle_state` is not `active` (rightsizing delta is not actionable for idle workloads).

GPU idle filter

Filter by GPU idle state (`active`, `idle`, or `zombie`; comma-separated values are ORed). GPU idle fields on container rows: `gpu_idle_state`, `gpu_idle_since`, `gpu_idle_duration_days`, `estimated_monthly_gpu_waste`.

```
GET /api/cost-management/v1/recommendations/openshift?filter[gpu_idle_state]=idle,zombie
```

Namespace idle filter

Filter namespaces by aggregated idle state (`active`, `idle`, or `zombie`).

```
GET /api/cost-management/v1/recommendations/openshift/namespaces?filter[idle_state]=idle,zombie
```

Container list ordering (idle)

Sort idle workloads with `order_by` and `order_how` (`asc` or `desc`):

```
GET /api/cost-management/v1/recommendations/openshift?order_by=idle_state
GET /api/cost-management/v1/recommendations/openshift?order_by=idle_duration_days
GET /api/cost-management/v1/recommendations/openshift?order_by=estimated_monthly_waste
```

PVC orphan idle

Orphaned PVC recommendations include `idle_since` and `idle_duration_days` fields tracking when zero usage was first detected.

Idle notification codes

Code	Name	When
5	IDLE_WORKLOAD	Container idle or zombie (`idle_state`)
8	ABANDONED_WORKLOAD	Legacy abandoned workload (all-zero usage when inline classification did not run)
15	NODE_IDLE	Node idle or zombie
26	GPU_IDLE	GPU idle or zombie

Async recalculation

PUT and DELETE on idle-detection settings enqueue async threshold recalculation for plugins: **container**, **gpu**, **namespace**, **node**, and **pvc**.

Savings summary by idle state

Group fleet idle waste by state (query value is ignored; use `*`):

```
GET /api/cost-management/v1/recommendations/openshift/savings-summary?group_by[idle_state]=*
```

Response (`FleetSavingsByIdleStateResponse`):

```
{
  "meta": { "count": 2 },
  "data": [
    {
      "idle_state": "zombie",
      "estimated_monthly_waste": { "value": "4200.00", "units": "USD" },
      "container_count": 12
    },
    {
      "idle_state": "idle",
      "estimated_monthly_waste": { "value": "8250.00", "units": "USD" },
      "container_count": 35
    }
  ]
}
```


Report rightsizing savings and idle waste separately on dashboards so totals are not double-counted (idle = eliminate workload; rightsizing = reduce requests).

Recommendation history and quality

See History and quality under OpenShift recommendations for fleet history, quality metrics, retention environment variables, and namespace history. VM per-name history is under VM recommendation history.

Forecasts

Cost forecasts predict future spending based on historical trends.

OpenShift cost forecast

```
GET /api/cost-management/v1/forecasts/openshift/costs/
```

AWS cost forecast

```
GET /api/cost-management/v1/forecasts/aws/costs/
```

Azure cost forecast

```
GET /api/cost-management/v1/forecasts/azure/costs/
```

Google Cloud cost forecast

```
GET /api/cost-management/v1/forecasts/gcp/costs/
```

OCP-on-cloud cost forecasts

```
GET /api/cost-management/v1/forecasts/openshift/infrastructures/all/costs/
GET /api/cost-management/v1/forecasts/openshift/infrastructures/aws/costs/
GET /api/cost-management/v1/forecasts/openshift/infrastructures/azure/costs/
GET /api/cost-management/v1/forecasts/openshift/infrastructures/gcp/costs/
```

Resource Types

Resource type endpoints return the available values for group_by and filter parameters. Useful for building dropdowns in UIs or discovering what filter values exist.

OpenShift

```
GET /api/cost-management/v1/resource-types/openshift-clusters/
GET /api/cost-management/v1/resource-types/openshift-projects/
GET /api/cost-management/v1/resource-types/openshift-nodes/
GET /api/cost-management/v1/resource-types/openshift-virtual-machines/
GET /api/cost-management/v1/resource-types/openshift-gpu-vendors/
GET /api/cost-management/v1/resource-types/openshift-gpu-models/
```

AWS

```
GET /api/cost-management/v1/resource-types/aws-accounts/
GET /api/cost-management/v1/resource-types/aws-services/
GET /api/cost-management/v1/resource-types/aws-regions/
GET /api/cost-management/v1/resource-types/aws-organizational-units/
GET /api/cost-management/v1/resource-types/aws-categories/
```

Azure

```
GET /api/cost-management/v1/resource-types/azure-subscription-guids/
GET /api/cost-management/v1/resource-types/azure-services/
GET /api/cost-management/v1/resource-types/azure-regions/
```

Google Cloud

```
GET /api/cost-management/v1/resource-types/gcp-accounts/
GET /api/cost-management/v1/resource-types/gcp-projects/
GET /api/cost-management/v1/resource-types/gcp-services/
GET /api/cost-management/v1/resource-types/gcp-regions/
```

Cost models

```
GET /api/cost-management/v1/resource-types/cost-models/
```

Cost Distribution Fields

OpenShift cost report responses include distributed overhead costs when a cost model has platform/worker distribution enabled:

Field	Definition
cost_platform_distributed	Platform project overhead distributed proportionally to user projects (by CPU or memory weighting)
cost_worker_unallocated_distributed	Worker node unallocated capacity distributed proportionally to user projects
cost_total	Total cost including raw + usage + markup + platform_distributed + worker_unallocated_distributed

These fields appear in nested form in the API JSON response (e.g. `cost.platform_distributed`, `cost.worker_unallocated_distributed`, `cost.total`).

Settings

Platform projects (cost groups)

```
GET /api/cost-management/v1/settings/cost-groups/
```

Enabled tags

All enabled tags:

```
GET /api/cost-management/v1/settings/tags/?filter[enabled]=true
```

Enabled tags filtered by source type:

```
GET /api/cost-management/v1/settings/tags/?filter[enabled]=true&filter[source_type]=OCP
GET /api/cost-management/v1/settings/tags/?filter[enabled]=true&filter[source_type]=AWS
GET /api/cost-management/v1/settings/tags/?filter[enabled]=true&filter[source_type]=Azure
GET /api/cost-management/v1/settings/tags/?filter[enabled]=true&filter[source_type]=GCP
```

Display currency

```
GET /api/cost-management/v1/account-settings/currency/
```

Cost models

List all cost models:

```
GET /api/cost-management/v1/cost-models/
```

Get rates on a single cost model by adding the cost model UUID:

```
GET /api/cost-management/v1/cost-models/{cost_model_uuid}/
```

Cost model relations are set at an OpenShift integration level (which includes one OpenShift cluster), or AWS/Azure/GCP integration level (which includes the parent and all child accounts in the same cloud integration). Everything covered by one integration shares the same cost model.

An integration can only be assigned to a single cost model, but multiple integrations (e. g. OpenShift clusters) can be assigned to the same cost model.

Matching data back to a cost model requires looking at the `source_uuid` value for each line item in the report endpoints, then searching for it in the cost-models endpoint.

Integrations

Formerly known as "sources", an integration is a cloud account or OpenShift cluster added to Cost Management. You can use the API to list sources added to your account or see last ingest completion times per source.

List all integrations:

```
GET /api/cost-management/v1/sources/
```

Get a specific integration:

```
GET /api/cost-management/v1/sources/{source_uuid}/
```

Check the stats on a particular integration:

```
GET /api/cost-management/v1/sources/{source_uuid}/stats/
```

Filtering and Pagination

Time scope filters

All cloud and OpenShift cost endpoints support the following filters:

`filter[resolution]`

`daily` or `monthly`. Sets report to daily or monthly breakdown.

`filter[time_scope_value]` and `filter[time_scope_units]`

Used together to set the scope value and unit for the returned data.

- `-10` or `-30` with `filter[time_scope_units]=day` for the last 10 or 30 days.
- `-1`, `-2`, or `-3` with `filter[time_scope_units]=month` for the last 1, 2, or 3 months.

You can also use explicit start and end dates:

```
?start_date=2022-09-01&end_date=2022-09-10
```

Exclude filter

Exclude specific values from results. Works with the same dimensions as `filter` and `group_by`:

```
GET /api/cost-
management/v1/reports/openshift/costs/?group_by[project]=*&exclude[project]=openshift-monitoring
GET /api/cost-management/v1/reports/aws/costs/?group_by[account]=*&exclude[account]=123456789012
```

Pagination

All list endpoints support `limit` and `offset` for pagination:

```
?limit=10&offset=0
```

The response `meta` object includes `count` (total items) and `links` includes `first`, `next`, `previous`, `last` URLs.

Get CSV or JSON Reports

Any of the report endpoints can return CSV format by setting the `Accept` header to `text/csv`.

Any of the report endpoints can return JSON format by setting the `Accept` header to `application/json`.

Current month aggregation by cluster:

```
curl --location -g \
  'https://console.redhat.com/api/cost-
management/v1/reports/openshift/costs/?filter[time_scope_units]=month&filter[time_scope_value]=-
1&filter[resolution]=monthly&group_by[cluster]=*' \
  -H 'accept: text/csv' \
  -H 'Authorization: Bearer ACCESS_TOKEN' \
  -o output.csv
```

Current month aggregation by project:

```
curl --location -g \
  'https://console.redhat.com/api/cost-
management/v1/reports/openshift/costs/?filter[time_scope_units]=month&filter[time_scope_value]=-
1&filter[resolution]=monthly&group_by[project]=*' \
  -H 'accept: text/csv' \
  -H 'Authorization: Bearer ACCESS_TOKEN' \
  -o output.csv
```

Current month grouped per tag:

```
curl --location -g \
  'https://console.redhat.com/api/cost-
management/v1/reports/openshift/costs/?filter[time_scope_units]=month&filter[time_scope_value]=-
1&filter[resolution]=monthly&group_by[tag:tag_key]=tag_value' \
  -H 'accept: text/csv' \
  -H 'Authorization: Bearer ACCESS_TOKEN' \
  -o output.csv
```

Current month combined group_by with filter (projects grouped, filtered by cluster and node):

```
curl --location -g \
  'https://console.redhat.com/api/cost-
management/v1/reports/openshift/costs/?filter[time_scope_units]=month&filter[time_scope_value]=-
1&filter[resolution]=monthly&group_by[project]=*&filter[cluster]=my_cluster&filter[node]=my_node'
\
  -H 'accept: text/csv' \
  -H 'Authorization: Bearer ACCESS_TOKEN' \
  -o output.csv
```

All of the above examples and additional parameter/filter options can be used with CSV export.

Basic Field Definitions

Field	Definition
sup_usage	Cost calculations from rates defined in the price list within a cost model when selecting Supplementary as the Calculation type
sup_raw	Not currently being used in the cost management backend
sup_markup	Markup set in the cost model, applied against sup_usage
sup_total	Total combined sup_cost: sup_usage + sup_markup
infra_usage	Cost calculations from rates defined in the price list within a cost model when selecting Infrastructure as the Calculation type
infra_raw	Cost coming directly from a cloud bill
infra_markup	Markup set in the cost model, applied against infra_raw and infra_usage

Field	Definition
infra_total	Total combined infra_cost: infra_usage + infra_markup + infra_raw
cost_usage	Total cost of usage: sup_usage + infra_usage
cost_raw	Total cost of raw: sup_raw + infra_raw
cost_markup	Total cost of markup: sup_markup + infra_markup
cost_total	Total combined cost: cost_usage + cost_markup + cost_raw
cost_units	Cost currency unit for all cost metrics
capacity	Total hourly capacity of metric (Memory/core hours)
capacity_count	Max cores/Mem for a cluster
capacity_count_units	Unit for count (e.g. Core)
usage	Total hourly usage for a project/node/cluster and/or day/month depending on the query
usage_units	Unit for the usage/request/limit metrics (e.g. Core-hours)
request	Total hourly request for a project/node/cluster and/or day/month depending on the query
request_unused	Total hourly request that is not being utilised (difference between request and usage)
request_unused_percent	Percent value for unused cluster request based on (unused_request / capacity) * 100
limit	Total hourly limit for a project/node/cluster and/or day/month depending on the query
gpu_request	Total GPU device requests for a project/node/cluster
gpu_limit	Total GPU device limits for a project/node/cluster
gpu_usage	Total GPU utilization for a project/node/cluster (in device-hours)